

REAL TIME THERMOSYPHON FLOW REVERSAL
PREDICTION THROUGH DATA ASSIMILATION OF COUPLED
COMPUTATIONAL FLUID DYNAMICS AND REDUCED
ORDER MODELING

A Thesis Presented

by

Andrew Reagan

to

The Faculty of the Graduate College

of

The University of Vermont

In Partial Fullfillment of the Requirements
for the Degree of Master of Science
Specializing in Applied Mathematics

January, 2013

Accepted by the Faculty of the Graduate College, The University of Vermont, in partial fulfillment of the requirements for the degree of Master of Science, specializing in Applied Mathematics.

Thesis Examination Committee:

Chris Danforth, Ph.D. Advisor

Peter Dodds, Ph.D.

Yves Dubief, Ph.D. Chairperson

Domenico Grasso, Ph.D. Dean, Graduate College

Date: November 15, 2013

Abstract

A thermal convection loop is a circular chamber filled with water, heated on the bottom half and cooled on the top half. With sufficiently great forcing of heat, the direction of fluid flow in the loop oscillates chaotically, thus forming an analog to the real weather system. Like full scale weather models, we can only observe a small part of state space, making prediction very difficult. To overcome this challenge, data assimilation methods and specifically ensemble methods use the computational model itself to estimate the uncertainty of our model to optimally combine these observations into an initial condition for predicting the future. First, we build and verify four distinct DA methods. Then, a computational fluid dynamic simulation of the loop and a reduced order model are both used by these DA methods to predict flow reversals. This happens in real time as experimental data is fed into the algorithm.

in dedication to

my parents, Donna and Kevin Reagan, for their unwavering support

Acknowledgements

I would like to thank professors Chris Danforth and Peter Dodds for their outstanding advising. I would like to thank my fellow students for all of their help and toleration through the past two years. And finally I would like to thank my girlfriend Sam Spisiak for knowing more than anyone else about this thesis.

Table of Contents

Dedication	ii
Acknowledgements	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction	1
1.2 History of NWP	2
1.3 IC determination: data assimilation	6
1.4 NWP and fluid dynamics: computational models	8
1.5 Toy climate experiments	9
2 Data Assimilation	11
2.1 Basic derivation, OI	12
2.2 3D-Var	18
2.3 Extended Kalman Filter	19
2.3.1 Coding the TLM	21
2.4 Ensemble Kalman Filter	23
2.4.1 Performace of EnKF, EKF on Lorenz 63	25
2.4.2 Ensemble Transform Kalman Filter (ETKF)	27
2.4.3 Local Ensemble Kalman Filter (LEKF)	29
2.4.4 Local Ensemble Transform Kalman Filter (LETKF)	31
Four Dimensional Local Ensemble Transform Kalman Filter (4D-LETKF)	34
2.5 4D-Var	34
2.5.1 Ensemble-variational hybrids	35
3 Computational Fluid Dynamics	37
3.1 Derivation of Governing Equations	37
3.1.1 Continuity Equation	37
3.1.2 Momentum Equation	39
3.1.3 Energy Equation	39
3.2 Discretization and Solving Algorithms	39

3.3 Solving with OpenFOAM	39
4 Thermal Convection Loop Experiment	41
4.1 Thermal Convection Loops	41
4.2 Ehrhard-Müller Equations	41
4.3 Detailed Experimental Setup	41
5 Results	43
5.1 Prediction Skill for Synthetic Data	43
5.1.1 Sampling frequency, distribution	43
5.1.2 Random error vs PMD perturbation scheme	43
5.1.3 Maintaining hydrostatic balance	44
5.2 Real-time Flow Reversal Prediction Skill	44
6 Conclusion	45
6.1 Main Result	45
6.1.1 More Details	45
Bibliography	46
Appendices	49
A Parameters	49
B Code	50

List of Figures

1.1	A schematic of the Energy Balance Model, a conceptual climate model based on the conservation of energy.	9
1.2	The “kitchen-sink” Earth System Model captures as many known processes as computationally feasible.	9
2.1	The Lorenz 63 attractor colored by the norm of the analysis increment. In other words, larger numbers represent where forecasts are poor, and the data assimilation algorithm puts more weight on the observation.	12
2.2	An explanation of how and why the best way to obtain a TLM is with a differentiated numerical scheme.	22
2.3	The future error predicted by the TLM is compared to the error growth in Lorenz 63 system for an initial perturbation with standard deviation of 0.1, averaged over 1000 TLM integrations. The ϵ is not the error predicted by the TLM, but rather the error of the TLM in predicting the error growth.	22
2.4	The difference of ensemble forecasts from the analysis is reported for 760 assimilation windows in one model run of length 200, with 10 ensemble members and an assimilation window of length 0.261. This has the same shape of as the difference between ensemble forecasts and the mean of the forecasts (I have this as well). This spread of ensemble forecasts is what allows us to estimate the error covariance of the forecast model, and appears fall from a normal distribution.	24
2.5	A sample timeseries of the ensembles used in the EnKF. In all tests, as seen here, 10 ensemble members are used.	24
2.6	For near-perfect initial knowledge of the atmospheric state, the RMS error of a prediction using DA is compared against a prediction that does not. Initial error is normal with mean 0 and variance 0.01, and the RMS errors reported are averages of 100 runs. Observational noise is normally distributed with mean 0 and variance 0.5, and an assimilation window length of 0.261 is used (approximately 5% of climatological variation). As would be expected, in the long run DA greatly improves forecast skill, as forecasts without DA saturate to climatological error. The initial period where a forecast exceeds the skill of DA is potentially due to the spin-up time required by the filter to dial in the analysis error covariance.	25
2.7	The RMS error (unscaled by climatology) is reported for our EKF and EnKF filters, measured as the difference between forecast and truth at the end of an assimilation window for the latter 2500 assimilation windows in a 3000 assimilation window model run. Error is measured in the only observed variable, x_1 . Increasing the assimilation window led to an decrease in predictive skill, as expected. Additive and multiplicative covariance inflation is the same as Harris et. al.	26
2.8	The RMS error averaged over 100 model runs of length 1000 windows is reported for the EKF for varying additive and multiplicative inflation factors Δ and μ . Each of the 100 model runs starts with a random IC, and the analysis forecast starts randomly. The filter performance RMS is computed as the RMS value of the difference between forecast and truth at the assimilation window for the latter 500 windows, allowing a spin-up of 500 windows. . .	27
2.9	The algorithm for the LEKF as described by Ott. Building on previous work, the algorithm is as one would expect.	31
3.1	A screenshot of the loop.	40
3.2	A snapshot of the mesh used for CFD simulations.	40

4.1	Professor Chris Danforth with the aluminum rig.	42
-----	---	----

List of Tables

2.1	Summary of Lorenz '63 parameters	34
A.1	EKF Covariance Inflation	49

Chapter 1

Introduction

In this chapter we explore the current state of numerical weather prediction, in particular data assimilation, computational fluid dynamics, and reduced order experiments.

1.1 Introduction

Prediction of the future state of many complex systems is integral to the functioning of our society. Some of these systems include weather (Hsiang et al. 2013), health (Ginsberg et al. 2008), the economy (Bollen et al. 2011, Sornette and Zhou 2006), marketing (Asur and Huberman 2010) and engineering (Savely et al. 1972). For weather in particular, this prediction happens on supercomputers across the world, as numerical weather model integrations taking our current best guess of the weather into the future. The accuracy of these predictions depend on the accuracy of the models themselves, and the quality of our current knowledge of the atmosphere.

Model accuracy has improved with better meteorological understanding of weather processes and advances in computing technology. To solve the initial value problem, techniques developed over the past 50 years are now broadly known as data assimilation. Formally, data assimilation is the process of using all available information, including short-range model forecasts and observations, to estimate the current state of a system as accurately as possible (Yang et al. 2006).

CHAPTER 1. INTRODUCTION

We employ a toy climate experiment as a testbed for improving numerical weather prediction algorithms, namely data assimilation methods. This approach is akin to the historical development of current methodologies, and provides a tractable system for rigorous analysis. The experiment is a thermal convection loop, which by design simplifies our problem into the prediction of mostly convection. The dynamics of thermal convection loops have been explored under both periodic (Keller) and chaotic (Welander 1995, Creveling et al. 1975, Gorman and Widmann 1984, Gorman et al. 1986, Ehrhard and Müller 1990, Yuen and Bau 1999, Jiang and Shoji 2003, Burroughs et al. 2005, Desrayaud et al. 2006, Yang et al. 2006, Ridouane et al. 2010) regimes. A full characterization of the computational behavior of a loop under flux boundary conditions by Louisos et. al. delimited four regimes: chaotic convection with reversals, high Ra aperiodic stable convection, steady stable convection, and conduction/quasi-conduction (Louisos et al. 2013). For the remainder of this work, we focus on the chaotic flow regime.

Computational simulations of the thermal convection loop are performed with the open-source finite volume C++ library OpenFOAM. The open-source nature of this software enable its integration with the data assimilation algorithms that were verified with a reduced order model.

1.2 History of NWP

The importance of Bjerknes in early developments in NWP is clearly described by Thompson’s “Charney and the revival of NWP” (Thompson 1990):

It was not until 1904 that Vilhelm Bjerknes - in a remarkable manifesto and testament of deterministic faith - started the central problem of NWP. This was the first explicit, coherent, recognition that the future state of the atmosphere is, *in principle*, completely determined by its detailed initial state and known boundary conditions, together with Newton’s equations of motion, the Boyle-Charles-Dalton equation of state, the equation of mass continuity, and the thermodynamic energy equation. Bjerknes went further: he outlined an ambitious, but logical program of observation, graphical analysis of meteorological data and graphical solution of the governing equations. He succeeded in persuading the Norwegians to support an expanded network of surface observation stations, founded the famous Bergen School of synoptic and dynamic meteorology, and ushered in the famous polar front theory of cyclone formation. Beyond providing a clear

CHAPTER 1. INTRODUCTION

goal and a sound physical approach to dynamical weather prediction, V. Bjerknes instilled his ideas in the minds of his students and their students in Bergen and in Oslo, three of whom were later to write important chapters in the development of NWP in the US (Rossby, Eliassen, and Fjortoft).

It was then until 1922 that Richardson suggested a practical way to solve these equations. From Jule G. Charney's "Dynamical forecasting by numerical process" (Charney 1951) we recognize Richardson's achievement:

He [Richardson] proposed to integrate the equations of motion numerically and showed exactly how this might be done. That the actual forecast used to test his method was unsuccessful was in no way a measure of the value of his work. In retrospect it becomes obvious that the inadequacies of observation alone would have doomed any attempt, however well conceived, a circumstance of which Richardson was aware. The real value of his work lay in the fact that it crystallized once and for all the essential problems that would have to be faced by future workers in the field and it laid down a thorough groundwork for their solution.

For a long time no one ventured to follow in Richardson's footsteps. The paucity of the observational network and the enormity of the computational task stood as apparently insurmountable barriers to the realization of his dream that one day it might be possible to advance the computation faster than the weather. But with the increase in the density and extent of the surface and upper-air observational network on the one hand, and the development of large-capacity high-speed computing machines on the other, interest has revived in Richardson's problem, and attempts have been made to attack it anew.

These efforts have been characterized by a devotion to objectives more limited than Richardson's. Instead of attempting to deal with the atmosphere in all its complexity, one tries to be satisfied with simplified models approximating the actual motions to a greater or lesser degree. By starting with models incorporating only what is thought to be the most important of the atmospheric influences, and by gradually bringing in the others, one is able to proceed inductively and thereby avoid the pitfalls inevitably encountered when a great many poorly understood factors are introduced all at once.

CHAPTER 1. INTRODUCTION

A necessary condition for the success of this stepwise method is, of course, that the first approximations bear a recognizable resemblance to the actual motions. Fortunately, the science of meteorology has progressed to the point where one feels that at least the main factors governing the large-scale atmospheric motion are well known. Thus integrations of even the linearized barotropic and thermally inactive baroclinic equations have yielded solutions bearing a marked resemblance to reality. At any rate, it seems clear that the models embodying the collective experience and the positive skill of the forecast cannot fail utterly. This conviction has served as the guiding principle in the work of the meteorology project at The Institute for Advanced Study [at Princeton University] with which the writer has been connected.

Richardson used a horizontal grid of about 200km, and 4 vertical layers in his mesh of Germany. Using the observations available, he computed the time derivative of pressure in the central box, predicting a change of 146 hPa whereas in reality there was almost no change. This huge discrepancy was due mostly to the fact that his initial conditions were not balanced, and therefore included fast-moving gravity waves which masked the true climate signal. If the integration had continued, it would have “exploded,” due to the Courant number induced by his mesh and timestep, which was greater than 1. We will revisit this condition in Chapter 3, but a simple explanation is as follows. The Courant number relates the movement of a quantity to the size of the mesh, written as

$$C = \frac{u_x \Delta t}{\Delta x}$$

in one dimension, and a value > 1 for a finite difference method solving partial differential conditions is divergent.

Early on, the problem of fast travelling waves from unbalanced IC was solved by filtering the equations of motion based on the quasi-geostrophic (slow time) balance. Although this made running models feasible on the computers of WW2, the incorporation of the full equations proved necessary for more accurate forecasts to be made.

The first computer model forecast was performed as a one day forecast on the ENIAC (the Electronic Numerical Integrator and Computer) by none other than J. von Neuman (Charney 1951), at the Aberdeen US

CHAPTER 1. INTRODUCTION

Army facility in Maryland. This incorporated von Neumann's "stored programming" technique (the ability to write "for" loops) and produced a forecast that remaining at least plausible.

Charney already saw the success of models with filtered equations of motion would not have enough accuracy and understood the difficulties that would be overcome in the next 50 years:

The discussion so far has dealt exclusively with the quasi-geostrophic equations as the basis for numerical forecasting. Yet there has been no intention to exclude the possibility that the primitive Eulerian equations can also be used for this purpose. The outlook for numerical forecasting would be indeed dismal if the quasi-geostrophic approximation represented the upper limit of attainable accuracy, for it is known that it applies only indifferently, if at all, to many of the small-scale but meteorologically significant motions.. We have merely indicated two obstacles that stand in the way of the applications of the primitive equations: first, there is the difficulty raised by Richardson that the horizontal divergence cannot be measured with significant accuracy. Moreover, the horizontal divergence is only one of a class of meteorological unobservables which also includes the horizontal acceleration. And second, if the primitive Eulerian equations are employed, a stringent and seemingly artificial bound is imposed on the size of the time interval for the finite difference equations. The first obstacle is the most formidable, for the second only means that the integration must proceed in the steps of the order of fifteen minutes rather than two hours. Yet the first does not seem insurmountable, as the following considerations will indicate.

What he describes is an integration where the initial fast-moving waves subside through the geostrophic adjustment of the model after a day, and do not otherwise affect the forecast. Charney concludes his 1950 paper with the following discussion of a problem that is still an active area of research, resolving subgrid-scale phenomena:

Nonadiabatic and frictional terms have been ignored in the body of the discussion because it was thought that one should first seek to determine how much of the motion could be explained without them. Ultimately they will have to be taken into account, particularly if the forecast period is to be extended to three or more days.

CHAPTER 1. INTRODUCTION

Condensational phenomena appear to be the simplest to introduce: one has only to add the equation of continuity for water vapor and replace the dry air by the moist adiabatic equation. Long-wave radiational effects can also be provided for, since our knowledge of the absorptive properties of water vapor and carbon dioxide has progressed to a point where quantitative estimates of radiational cooling can be made, although the presence of clouds will complicate the problem considerably.

The most difficult phenomena to include have to do with the turbulent transfer of momentum and heat. A great deal of research remains to be done before enough is known about these effects to permit the assignment of even rough values to the eddy coefficients of viscosity and heat conduction. Owing to their statistically indeterminate nature, the turbulent properties of the atmosphere place an upper limit to the accuracy obtainable by dynamical methods of forecasting, beyond which we shall have to rely upon statistical methods. But it seems certain that much progress can be made before these limits can be reached.

The shortcomings of our understanding of the subgrid-scale phenomena do impose a limit on our ability to predict the weather, but this is not the upper limit. The upper limit of predictability exists for even a perfect model, as E. Lorenz shows in (Lorenz 1963) with a three variable model that exhibits sensitive dependence on initial conditions.

1.3 IC determination: data assimilation

The purpose of data assimilation is defined by Talagrand (Talagrand 1997) as “using all the available information, to determine as accurately as possible the state of the atmospheric (or oceanic) flow.” Broadly, data assimilation algorithms are used from areas as disparate as quadcopter stabilization (Achtelik et al. 2009) to finance (?).

Data assimilation algorithms work on predict, observe, and assimilate cycle. Formally, the data assimilation problem is solved by minimizing the error in the presence of the above constraints. The prediction step involves making a prediction of the future state of the system, as well as the error of the model in some capacity. Observing various systems comes in many flavors: rawindsones and satellite irradiance for the atmosphere, temperature and velocity reconstruction of experiments from simple sensors, or sampling the

CHAPTER 1. INTRODUCTION

market in finance. Assimilation is the combination of these observations and the predictive model in such a way that minimizes the error of the future prediction based on this analysis.

The difficulties of this process arise in many fashions, and are addressed by parameter adjustments of existing algorithms or new approaches altogether. The first problem is ensuring the analysis respects the core balances of the model. In the atmosphere this is the geostrophic balance, and in other systems this can arise in the conservation of physical (or otherwise) quantities.

In real applications, observations are made at timesteps that are not always in agreement with the analysis increment and can be made more frequently. However the incorporation of more observations will not always improve forecasts: in the case where these observation can only be incorporated at the assimilation time the forecasts can degrade in quality (Kalnay et al. 2007). Algorithms which are capable of assimilating observations at the time they are made have a distinct advantage in this respect. In addition, the observations are not always direct measurements of the variables of the model. This can create non-linearities in the observation operator, the function that transforms observations in the physical space into model space. In Chapter 2 we will see where a non-linear operator complicates the minimization of the analysis error.

Models aren't perfect. The incorporation of model bias into the assimilation becomes important with imperfect models, which for the weather all of our models are, and although this issue is well-studied it is a constant concern.

The final difficulty we need to address is the computational cost of the various algorithms. The complexity of models creates issues for both the computation speed and storage requirements, an example of each follows. The direct computation of the model covariance can be performed by integrating a linear tangent model which propagates errors in the model forward in time, but integrating this model amounts to n^2 model integrations where n is the number of variables in the model. For large systems, this is well beyond reach. Storing a covariance matrix at the full rank of model variables is not possible even for a model of the experiment we will address, which has 216,000 model variables. Workarounds for these specific problems include approximating the covariance without updating it or updating it using an ensemble of model runs, and reformulating the assimilation problem such that the covariance is only computed in observation space, which is orders of magnitude smaller in general.

1.4 NWP and fluid dynamics: computational models

Derived from Newton’s law of motion, the basic equations governing flow of a viscous, heat-conducting fluid have been known since as early as 1823 (Navier 1823) and in their current form since 1846 (Stokes 1846). These are the Navier-Stokes equations. For the atmosphere, we can make various approximations to simplify the problem to arrive at what are known of equations of Geophysical Fluid Dynamics, which differ mainly in the incorporation of the Coriolis effect and the neglecting of vertical components of advection. In both cases, the principle equations is the momentum equation, supplemented by the continuity equation and the energy.

As stated, my interest here is not the accuracy of computational fluid dynamic simulations themselves, but rather their incorporation into the prediction problem. For this reason, the depth of exploration of available models is limited.

We chose to perform simulations in the open-source C++ library OpenFOAM. Because the code is open-source I can modify it as necessary for data assimilation, and of particular importance becomes the discretized grid.

There are two main directions in which I do explore the computational model: meshing and solving. Creating the mesh is the discretization of the governing equations. The importance of properly designing and characterizing the mesh is profound: without the use of unreliable turbulence models, and without a scale small enough to resolve all of the scales of turbulence, we inevitably fail to capture the effects of sub-grid scale phenomena. In weather models, this is a known problem and correcting for these is an open area of research. Turbulence models include $k - \epsilon$ models, LES, and laminar (dummy) models.

Secondly, the choice of solving technique is a source of aleatoric uncertainty. The schemes which OpenFOAM has available are known to be dissipative. That is, small perturbations such as those from a random error perturbation to an ensemble member in the EnKF are damped away. Without properly correcting this problem, the ensemble does not adequately capture the uncertainties in observations to obtain an approximation of the model error growth.

We investigate these meshing and solving techniques in greater detail in Chapter 3.

1.5 Toy climate experiments

Mathematical and computational models span from simple to kitchen-sink, and both have their place. Simple, or “toy”, models and experiments are important to gain theoretical understanding of the key processes at play. An example of such a conceptual model used to study climate is the energy balance model, pictured below.

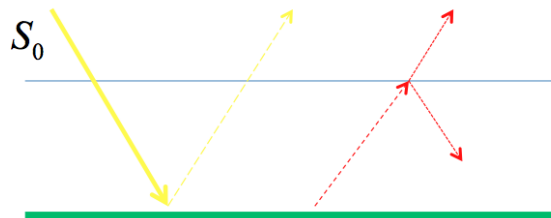


Figure 1.1: A schematic of the Energy Balance Model, a conceptual climate model based on the conservation of energy.

With a simple model it is possible to capture the key properties of simple concepts (e.g. the greenhouse effect in the above energy balance model) and for this reason they have been and continue to be employed throughout scientific disciplines. The “kitchen-sink” models, like the Earth System Models (ESMs) used to study climate, incorporate all of the known physical, chemical, and biological processes for the fullest picture of the system. In this case, we are living in our experiment.

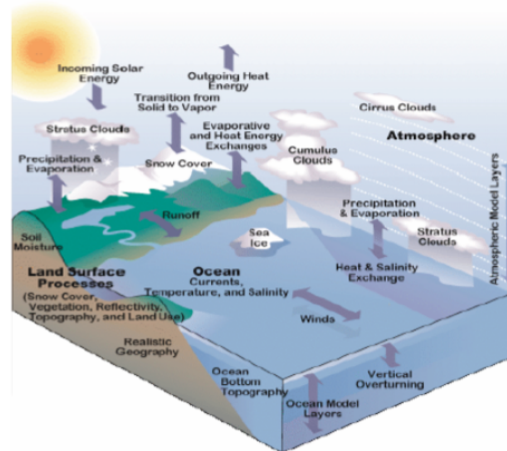


Figure 1.2: The “kitchen-sink” Earth System Model captures as many known processes as computationally feasible.

CHAPTER 1. INTRODUCTION

We study a model that is more complex than the EBM but is still analytically tractable, known as the Lorenz 1963 system (Lorenz 63 for short). In 1962, Saltzman attempted to make weather predictions by reducing the equations of motion of the atmosphere into their core processes (Saltzman 1962). Then in 1963 Edward Lorenz reduced this system even further to 3 equations, and used them to demonstrate his landmark discovery of deterministic non-periodic flow (Lorenz 1963). These equations have since been the subject of intense study and have changed the way we view prediction and determinism, remaining the simple system of choice for examining nonlinear behavior today (Kalnay et al. 2007).

Thermal convection loops are one of the few known experimental implementations of Lorenz's 1963 system of equations. Put simply, a thermal convection loop is a hula-hoop shaped tube of water, heated on the bottom half and cooled on the top half. As a nonmechanical heat pump, thermal convection loops have found applications for heat transfer in solar hot water heaters (Belessiotis and Mathioulakis 2002), cooling systems for computers (Beitelmal and Patel 2002), roads and railways that cross permafrost (Cherry et al. 2006), nuclear power plants (Detman 1968, Beine et al. 1992, Kwant and Boardman 1992), and other industrial applications. Our particular experiment consists of a thick-walled plastic tube, heated on the bottom by a constant temperature water bath, and cooled on top by room temperature air.

Varying the forcing of temperature applied to the bottom half of the loop there are three experimentally accessible flow regimes of interest to us: conduction, steady convection, and unsteady convection. The latter regime exhibits the much studied phenomenon of chaos in its changes of rotational direction. This regime can be described by the Lorenz 1963 system with the canonical parameter values listed in 2.1. Edward Lorenz discovered and studied chaos most of his life, and it said to have jotted this simple description on a visit to the University of Maryland:

Chaos: the approximate past does not approximately determine the future.

In chapter 4, we present a derivation of Ehrhard-Müller Equations governing the flow in the loop following the work of (Harris et al. 2011). These equations are akin to the Lorenz 63 system, as we shall see, and the experimental implementation is described in more detail in Chapter 4 as well.

Chapter 2

Data Assimilation

The goal of this chapter is to introduce and detail the practical implementation of state of the art data assimilation algorithms. I derive the general form for the problem, and then talk about how each filter works. For a selection of these algorithms, I illustrate some of their properties via a MATLAB implementation on the Lorenz 1963 three-variable model:

$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= rx - y - xz \\ \frac{dz}{dt} &= xy - bz\end{aligned}$$

The canonical choice of $\sigma = 10$, $b = 8/3$ and $r = 28$ is used for all tests here, which produce the well known butterfly attractor (Figure 2.1):

From the simpler examples, I build the most advanced 4D-Var and 4DLETKF approaches with the full CFD model (216K variables).

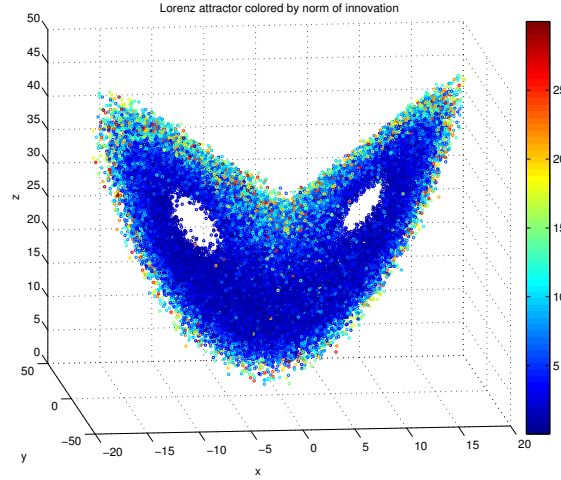


Figure 2.1: The Lorenz 63 attractor colored by the norm of the analysis increment. In other words, larger numbers represent where forecasts are poor, and the data assimilation algorithm puts more weight on the observation.

2.1 Basic derivation, OI

I begin by deriving the equations for OI and for the Kalman Filter with one scalar observation. The goal of my derivation is to highlight which statistical information is useful, which is necessary, and what assumptions are generally made in for formulation of more complex algorithms.

Beginning the one variable U , which could represent the horizontal velocity at one gridpoint, say we make two independent observations of U : U_1 and U_2 . If the true state of U is given by U_t , then the goal is to produce an analysis of $U_{1,2}$ which we call U_a , that is our best guess of U_t .

Denote the errors of $U_{1,2}$ by $\epsilon_{1,2}$ respectively, such that

$$T_{1,2} = T_t + \epsilon_{1,2}.$$

For starters, we assume that we do not know $\epsilon_{1,2}$. Next, assume that the errors vary around the truth, such that they are unbiased. This is written as

$$E(T_{1,2} - T_t) = E(\epsilon_{1,2}) = 0.$$

CHAPTER 2. DA

As of yet, our best guess of U_t would be the average $(U_1 + U_2)/2$, and for any hope of knowing U_t well we would need many unbiased observations at the same time. This is unrealistic, and to move forward we assume that we know the variance of each $T_{1,2}$. With this knowledge, which we write

$$E[\epsilon_{1,2}^2] = \sigma_{1,2}^2 \quad (2.1)$$

and further assume that these errors are uncorrelated

$$E[\epsilon_1 \epsilon_1] = 0 \quad (2.2)$$

which in the case of observing the atmosphere is at best hopeful.

We again attempt a linear fit of these observations to the truth, and can do better than an average. For U_a our best guess of U_t we write this linear fit as

$$U_a = \alpha_1 T_1 + \alpha_2 T_2. \quad (2.3)$$

If U_a is to be our best guess, we desire it to unbiased

$$E(U_a) = E(U_t) \Rightarrow \alpha_1 + \alpha_2 = 1$$

and to have minimal variance error σ_a^2 .

At this point note all of the assumptions that we have made: independent observations, unbiased instruments, knowledge of error variance, and uncorrelated observation errors. Of these, knowing the error variances (and in multiple dimensions, error covariance matrices) will become our focus and better solutions to determining this have been largely responsible for major improvements in forecast skill.

Solving 2.3 by substituting $\alpha_2 = 1 - \alpha_1$ have

$$\begin{aligned} \sigma_a^2 &= E[(\alpha_1(U_1 - U_t) + (1 - \alpha_1)(U_2 - U_t))^2] \\ &= \alpha_1^2 E[\epsilon_1^2] + (1 - \alpha_1^2) E[\epsilon_2^2] + 2\alpha_1(1 - \alpha_1) E[\epsilon_1 \epsilon_2] \\ &= \alpha_1^2 \sigma_1^2 + (1 - \alpha_1^2) \sigma_2^2 \end{aligned}$$

CHAPTER 2. DA

To minimize σ_a^2 with respect to α_1 we compute $\frac{\partial}{\partial \alpha_1}$ of the above and solve for 0 to obtain that

$$\alpha_1 = \frac{\sigma_2^2}{2(\sigma_2^2 - \sigma_1^2)}$$

and similarly for α_2 .

We can also state the relationship for σ_a^2 as a function of $\sigma_{1,2}^2$ by substituting our form for α_1 into the second step of 2.3 to obtain:

$$\frac{1}{\sigma_a^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}. \quad (2.4)$$

This last formula says that given accurate statistical knowledge of $U_{1,2}$, the precision of the analysis is the sum of the precisions of the measurements.

We can now formulate the simplest sequence data assimilation cycle by using our forecast model to generate a background forecast U_b and setting $U_1 = U_b$. We then assimilate the observations U_o by setting $U_2 = U_o$ and solving for the analysis velocity U_a as

$$U_a = U_b + W(T_o - T_b). \quad (2.5)$$

The difference $T_o - T_b$ is commonly referred to as the “innovation” and represents the new knowledge from the observations with respect to the background forecast. The optimal weight W is given by 2.4 then as

$$W = \frac{\sigma_b^2}{\sigma_b^2 - \sigma_o^2} = \sigma_b^2(\sigma_b^2 - \sigma_o^2)^{-1}. \quad (2.6)$$

The analysis error variance and precision are defined as before, and for convenience we can rewrite this in terms of W as (Kalnay 2003):

$$\sigma_a^2 = (1 - W)\sigma_b^2. \quad (2.7)$$

We now generalize the above for models of more than one variable to obtain the full equations for OI. This was originally done by (for Aerology et al. 1960), and independently by (Gandin 1965). The following notation will be used for the OI and the other multivariate filters. Consider a field of grid points, where there may be $\mathbf{x}_i = (p, p_{\text{rgh}}, T, u_x, u_y, u_z)$ variables at each point for pressure, hydrostatic pressure, temperature, and three dimensions of velocity. We combine all the variables into one list $\mathbf{x}$ of length n where n is the

CHAPTER 2. DA

product of the number of grid points of the number of model variables. This list is ordered by grid point and then by variable (e.g. $\mathbf{x} = (p_1, T_1, p_2, T_2, \dots, p_{n/2}, T_{n/2})$ for a field with just variables pressure p and temperature T). Denote the observations as $\mathbf{y}_0$, where the ordering is the same as $\mathbf{x}$ but the dimension is $|\mathbf{y}_0| \leq |\mathbf{x}|$ and the observations are not strictly at the grid points of $\mathbf{x}$ or even direct measurements of the variables in $\mathbf{x}$. In our experiment, temperature measurements are in the model variable T , but are understood as the average of the temperature of the grid point surrounding it within some chosen radius. Examples of indirect and spatially heterogenous observations in model weather forecasting are radar reflectivities and Doppler shifts, satellite radiances, and global positioning system (GPS) atmospheric refractivities (Kalnay 2003).

We can now write the write that analysis with similar notation as for the scalar case (no additional explanation is necessary) with the addition of the observation operator H which transforms model variables into observation space:

$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{W}\mathbf{d} \quad (2.8)$$

$$\mathbf{d} = \mathbf{y}_0 - H(\mathbf{x}_b) \quad (2.9)$$

$$\epsilon_a = \mathbf{x}_t - \mathbf{x}_a. \quad (2.10)$$

Note that it is advatageous to write H as an $n_{\text{obs}} \times n$ matrix, where H is not required to be linear.

Again, our goal is to obtain the weight matrix $\mathbf{W}$ that minimizes the analysis error ϵ_a . In Kalman Filtering, the weight matrix $\mathbf{W}$ is called the gain matrix $\mathbf{K}$. First, we assume that the background and observations are unbiased.

$$E[\epsilon_b] = E[\mathbf{x}_b] - E[\mathbf{x}_t] = 0$$

$$E[\epsilon_o] = E[\mathbf{y}_o] - E[H(\mathbf{x}_t)] = 0.$$

This a reasonable assumption, and if it does not hold true we can account for this as a part of the analysis cycle (Dee and Da Silva 1998).

CHAPTER 2. DA

Define the forecast error covariances as

$$\mathbf{P}_a = \mathbf{A} = E [\epsilon_a \epsilon_a^T] \quad (2.11)$$

$$\mathbf{P}_b = \mathbf{B} = E [\epsilon_b \epsilon_b^T] \quad (2.12)$$

$$\mathbf{P}_o = \mathbf{R} = E [\epsilon_o \epsilon_o^T]. \quad (2.13)$$

Linearize the observation operator

$$H(\mathbf{x} + \partial\mathbf{x}) = H(\mathbf{x}) + \mathbf{H}\partial\mathbf{x} \quad (2.14)$$

where $h_{ij} = \partial H_i / \partial x_j$ are the entries of $\mathbf{H}$. Since the background is a decent forecast, it is convenient to write our equations in terms of changes to $\mathbf{x}_b$. The observational increment $\mathbf{d}$ is thus

$$\mathbf{d} = \mathbf{y}_o - H(\mathbf{x}_b) = \mathbf{y}_o - H(\mathbf{x}_t + (\mathbf{x}_b - \mathbf{x}_t)) \quad (2.15)$$

$$= \mathbf{y}_o - H(\mathbf{x}_t) - \mathbf{H}(\mathbf{x}_b - \mathbf{x}_t) = \epsilon_o - \mathbf{H}\epsilon_b. \quad (2.16)$$

To solve for the weight matrix $\mathbf{W}$ assume that we know $\mathbf{B}$ and $\mathbf{R}$, and that their errors are uncorrelated:

$$E [\epsilon_o \epsilon_b^T] = E [\epsilon_b \epsilon_o^T] = 0. \quad (2.17)$$

We now use the method of best linear unbiased estimation (BLUE) to derive $\mathbf{W}$ in the equation $\mathbf{x}_a - \mathbf{x}_b = \mathbf{W}\mathbf{d}$ which approximates $\mathbf{x}_a - \mathbf{x}_b = \mathbf{W}\mathbf{d} - \epsilon_a$. Since $\mathbf{d} = \epsilon_o - \mathbf{H}\epsilon_b$ and the well known BLUE solution to this simple equation, the $\mathbf{W}$ that minimizes $\epsilon_a \epsilon_a^T$ is given by

$$\mathbf{W} = E[(\mathbf{x}_t - \mathbf{x}_b)(\mathbf{y}_o - \mathbf{H}\mathbf{x}_b)^T] (E[(\mathbf{y}_o - \mathbf{H}\mathbf{x}_b)(\mathbf{y}_o - \mathbf{H}\mathbf{x}_b)^T])^{-1} \quad (2.18)$$

$$= E[(-\epsilon_b)(\epsilon_o - \mathbf{H}\epsilon_b)^T] (E[(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_o - \mathbf{H}\epsilon_b)^T])^{-1} \quad (2.19)$$

$$= E[-\epsilon_b\epsilon_o^T + \epsilon_b\epsilon_b^T\mathbf{H}^T] (E[\epsilon_o\epsilon_o^T - \mathbf{H}\epsilon_b\epsilon_o^T - \epsilon_o\epsilon_b^T\mathbf{H}^T + \mathbf{H}\epsilon_b\epsilon_b^T\mathbf{H}^T])^{-1} \quad (2.20)$$

$$= (E[-\cancel{\epsilon_b\epsilon_o^T}] + E[\epsilon_b\epsilon_b^T\mathbf{H}^T]) (E[\epsilon_o\epsilon_o^T] - \cancel{\mathbf{H}E[\epsilon_b\epsilon_o^T]} - \cancel{E[\epsilon_o\epsilon_b^T]\mathbf{H}^T} + \mathbf{H}E[\epsilon_b\epsilon_b^T\mathbf{H}^T])^{-1} \quad (2.21)$$

$$= \mathbf{B}\mathbf{H}^T (\mathbf{R} + \mathbf{H}\mathbf{B}\mathbf{H}^T)^{-1} \quad (2.22)$$

Now we solve for the analysis error covariance $\mathbf{P}_a$ using $\mathbf{W}$ from above and $\epsilon_a = \epsilon_b + \mathbf{W}\mathbf{d}$. This is (with the final bit of sneakiness using $\mathbf{W}\mathbf{R} - \mathbf{B}\mathbf{H}^T = -\mathbf{W}\mathbf{H}\mathbf{B}\mathbf{H}^T$):

$$\mathbf{P}_a = E[\epsilon_a\epsilon_a^T] = E[(\epsilon_b + \mathbf{W}\mathbf{d})(\epsilon_b + \mathbf{W}\mathbf{d})^T] \quad (2.23)$$

$$= E[(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))^T] \quad (2.24)$$

$$= E[(\epsilon_b + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b))(\epsilon_b^T + (\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T)] \quad (2.25)$$

$$= E[\epsilon_b\epsilon_b^T + \epsilon_b(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b)\epsilon_b^T + \mathbf{W}(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)\mathbf{W}^T] \quad (2.26)$$

$$= E[\epsilon_b\epsilon_b^T] + E[\epsilon_b(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)]\mathbf{W}^T + \mathbf{W}E[(\epsilon_o - \mathbf{H}\epsilon_b)\epsilon_b^T] + \mathbf{W}E[(\epsilon_o - \mathbf{H}\epsilon_b)(\epsilon_o^T - \epsilon_b^T\mathbf{H}^T)]\mathbf{W}^T \quad (2.27)$$

$$= \mathbf{B} - \mathbf{B}\mathbf{H}^T\mathbf{W}^T - \mathbf{W}\mathbf{H}\mathbf{B} + \mathbf{W}E[\epsilon_o\epsilon_o^T - \cancel{\epsilon_o\epsilon_b^T\mathbf{H}^T} - \cancel{\mathbf{H}\epsilon_b\epsilon_o^T} + \mathbf{H}\epsilon_b\epsilon_b^T\mathbf{H}^T]\mathbf{W}^T \quad (2.28)$$

$$= \mathbf{B} - \mathbf{B}\mathbf{H}^T\mathbf{W}^T - \mathbf{W}\mathbf{H}\mathbf{B} + \mathbf{W}\mathbf{R}\mathbf{W} + \mathbf{W}\mathbf{H}\mathbf{B}\mathbf{H}^T\mathbf{W}^T \quad (2.29)$$

$$= \mathbf{B} - \mathbf{W}\mathbf{H}\mathbf{B} + (\mathbf{W}\mathbf{R} - \mathbf{B}\mathbf{H}^T)\mathbf{W}^T + \mathbf{W}\mathbf{H}\mathbf{B}\mathbf{H}^T\mathbf{W}^T \quad (2.30)$$

$$= (\mathbf{I} - \mathbf{W}\mathbf{H})\mathbf{B} \quad (2.31)$$

Note that in this formulation, we have assumed that $\mathbf{R}$ (the observation error covariance) accounts for both the instrumental error and the “representativeness” error, a term coined by Lorenc to describe the fact that the model does not account for subgrid-scale variability (Lorenc 1981). To summarize, the equations for OI are

$$\mathbf{W} = \mathbf{B}\mathbf{H}^T (\mathbf{R} + \mathbf{H}\mathbf{B}\mathbf{H}^T)^{-1} \quad (2.32)$$

$$\mathbf{P}_a = (\mathbf{I} - \mathbf{W}\mathbf{H})\mathbf{B} \quad (2.33)$$

As a final note, it is also possible to derive the “precision” of the analysis $\mathbf{P}_a^{-1}$ but in practice it is impractical to compute the inverse of the full matrix $\mathbf{P}_a$, so I omit this derivation. In general is also not possible to solve for $\mathbf{W}$ for the whole model, and localization of the solution is achieved by considering each grid point and the observations within some set radius of this. For different models, this equates to different assumptions on the correlations of observations, which I do not consider here.

2.2 3D-Var

Simply put, 3D-Var is the variational (cost-function) approach to finding the analysis. It has been shown that 3D-var solves the same statistical problem as OI (Lorenc 1986). The usefulness of the variational approach comes from the computational efficiency. This is why the 3D-Var analysis scheme became the most widely used data assimilation technique in operational weather forecasting, only to be supplemented by 4D-Var today.

The formulation of the cost function can be done in a Bayesian framework (Purser 1984) or using maximum likelihood (ML) approach (below). First, it is not difficult to see that solving for $\partial J / \partial U = 0$ for $U = U_a$ where we define $J(U)$ as

$$J(T) = \frac{1}{2} \left[\frac{(U - U_1)^2}{\sigma_1^2} + \frac{(U - U_2)^2}{\sigma_2^2} \right] \quad (2.34)$$

gives the same solution as obtained through least squares in our scalar assimilation. The formulation of this cost function obtained through the ML approach is simple (Edwards 1984). Consider the likelihood of true value U given an observation U_1 with standard deviation σ_1 :

$$L_{\sigma_1}(U|U_1) = p_{\sigma_1}(U_1|U) = \frac{1}{\sqrt{2\pi}\sigma_1} e^{-\frac{(U_1 - U)^2}{2\sigma_1^2}}. \quad (2.35)$$

The most likely value of U given two observations $U_{1,2}$ is the value which maximizes the joint likelihood (the product of the likelihoods), hence maximizes the logarithm of the joint likelihood (since the logarithm is increasing):

$$\max_U \ln L_{\sigma_1, \sigma_2}(U|U_1, U_2) = \max_U \left[a - \frac{(U - U_1)^2}{\sigma_1^2} - \frac{(U - U_2)^2}{\sigma_2^2} \right] \quad (2.36)$$

and this is equivalent to minimizing the cost function J above.

Jumping straight to result of Lorenc (Lorenc 1986) (we could define likelihood functions just like above), we define the multivariate 3D-Var as finding the $\mathbf{x}_a$ that minimizes the cost function

$$J(\mathbf{x}) = (\mathbf{x} - \mathbf{x}_b)^T \mathbf{B}^{-1} (\mathbf{x} - \mathbf{x}_b) + (\mathbf{y}_o - H(\mathbf{x}))^T \mathbf{R} (\mathbf{y}_o - H(\mathbf{x})). \quad (2.37)$$

The formal minimization of J can be obtained by solving for $\nabla_x J(\mathbf{x}_a)$, but since this is not useful in practice (iterative methods are used), we only present the result from (Kalnay 2003):

$$\mathbf{x}_a = \mathbf{x}_b + (\mathbf{B}^{-1} + \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H})^{-1} \mathbf{H}^T \mathbf{R}^{-1} (\mathbf{y}_o - H(\mathbf{x}_b)). \quad (2.38)$$

The PSAS data assimilation is so similar to 3D-Var that I do not derive it here, the only difference being that the cost function J is solved in observation space. Since there are typically less observations than there are model variables, this is a more efficient method.

Finally I note for 3D-Var that operationally $\mathbf{B}$ can be estimated with some flow dependence with the NMC or NCEP method, i.e.

$$\mathbf{B} \approx \alpha E [(\mathbf{x}_f(48h) - \mathbf{x}_f(24h))(\mathbf{x}_f(48h) - \mathbf{x}_f(24h))^T] \quad (2.39)$$

with on the order of 50 different short range forecasts.

2.3 Extended Kalman Filter

The Extended Kalman Filter (EKF) is the “gold standard” of data assimilation methods. It is the Kalman Filter “extended” for application to a nonlinear model. It works by updating our knowledge of error covariance matrix for each assimilation window, using a Tangent Linear Model (TLM), whereas in a traditional Kalman

CHAPTER 2. DA

Filter the model itself can update the error covariance. The TLM is precisely the model (written as a matrix) that transforms a perturbation at time t to a perturbation at time $t + \Delta t$, analytically equivalent to the Jacobian of the model. Using the notation of Kalnay (Kalnay 2003), this amounts to making a forecast with the nonlinear model M , and updating the error covariance matrix $\mathbf{P}$ with the TLM L , and adjoint model L^T

$$\begin{aligned}\mathbf{x}^f(t_i) &= M_{i-1}[\mathbf{x}^a(t_{i-1})] \\ \mathbf{P}^f(t_i) &= L_{i-1}\mathbf{P}^a(t_{i-1})L_{i-1}^T + \mathbf{Q}(t_{i-1})\end{aligned}$$

where $\mathbf{Q}$ is the noise covariance matrix (model error). In the experiments with Lorenz 63 presented in this section, $\mathbf{Q} = 0$ since our model is perfect.

The analysis step is then written as (for H the observation operator):

$$\mathbf{x}^a(t_i) = \mathbf{x}^f(t_i) + \mathbf{K}_i \mathbf{d}_i \quad (2.40)$$

$$\mathbf{P}^a(t_i) = (\mathbf{I} - \mathbf{K}_i \mathbf{H}_i) \mathbf{P}^f(t_i) \quad (2.41)$$

where

$$\mathbf{d}_i = \mathbf{y}_i^o - \mathbf{H}[x^f(t_i)]$$

is the innovation.

The Kalman gain matrix is computed to minimize the analysis error covariance P_i^a as

$$\mathbf{K}_i = \mathbf{P}^f(t_i) \mathbf{H}_i^T [\mathbf{R}_i + \mathbf{H}_i \mathbf{P}^f(t_i) \mathbf{H}_i^T]^{-1}$$

where $\mathbf{R}_i$ is the observation error covariance.

Since we are making observations of the truth with random normal errors of standard deviation ϵ , then $\mathbf{R}$ is a matrix of just ϵ values.

The most difficult, and most computationally expensive, part of the EKF is deriving and integrating the TLM. For this reason, no operational weather centers use the EKF, although approximations such as the EnKF are promising (next).

2.3.1 Coding the TLM

Actually coding the TLM was difficult, so I'll try to spell it out here, to make sure I understand it. The TLM is the model which advances an initial perturbation $\delta \mathbf{x}_i$ at timestep i to a final perturbation $\delta^* \mathbf{x}_{i+1}$ at timestep $i + 1$. The dynamical system we are interested in, Lorenz '63, is given as a system of ODE's:

$$\frac{d\mathbf{x}}{dt} = F(\mathbf{x}).$$

We integrate this system using a numerical scheme of our choice (here I used RK2), to obtain a model M discretized in time.

$$\mathbf{x}(t) = M[\mathbf{x}(t_0)].$$

Introducing a small perturbation $\mathbf{y}$, we can approximate the our model M of $\mathbf{x}(t_0) + \mathbf{y}(t_0)$ with a Taylor series around $\mathbf{x}(t_0)$, for which we assume to already have solved for:

$$\begin{aligned} M[\mathbf{x}(t_0) + \mathbf{y}(t_0)] &= M[\mathbf{x}(t_0)] + \frac{\partial M}{\partial \mathbf{x}} \mathbf{y}(t_0) + O[\mathbf{y}(t_0)^2] \\ &\approx \mathbf{x}(t) + \frac{\partial M}{\partial \mathbf{x}} \mathbf{y}(t_0). \end{aligned}$$

We can then solve for the linear evolution of the small perturbation $\mathbf{y}(t_0)$ as

$$\frac{d\mathbf{y}}{dt} = \mathbf{J}\mathbf{y}$$

where $\mathbf{J} = \partial F / \partial \mathbf{x}$ is the Jacobian of F . We can solve the above system of linear ordinary differential equations using the same numerical scheme as we did for the nonlinear model.

The problem with solving that system of equations is that the Jacobian matrix of discretized code is not necessarily identical to the discretization of the Jacobian operator for the analytic system. This is a problem because we need to have the TLM of our model M , which is the time-space discretization of the solution to $d\mathbf{x}/dt = F(\mathbf{x})$. We can apply our numerical method to the $d\mathbf{x}/dt = F(\mathbf{x})$ to obtain M explicitly, and then take the Jacobian of this. But, that is extremely messy (and costly), since recall the RK4 method had many nested function iterations. It is therefore desirable to take the derivative of the numerical scheme directly, and apply this differentiated numerical scheme to the system of equations $F(\mathbf{x})$ to obtain the TLM. A schematic

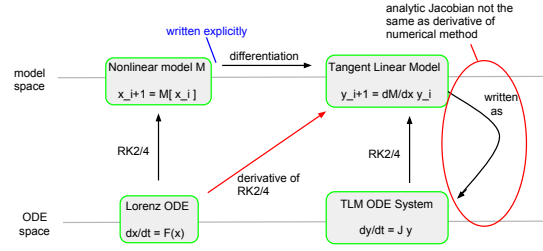


Figure 2.2: An explanation of how and why the best way to obtain a TLM is with a differentiated numerical scheme.

of this scenario is illustrated in Figure 2.2. It is for this reason that there exists code that takes the derivative of numerical code (i.e. Fortran) for implementing the EKF on models larger than 3 dimensions with numerical schemes more complicated than RK2 or RK4.

In order to verify my implementation of the TLM, I propagate a small error in the Lorenz 63 system and plot the difference between that error and the TLM predicted error in each variable forward in time in Figure 2.3.

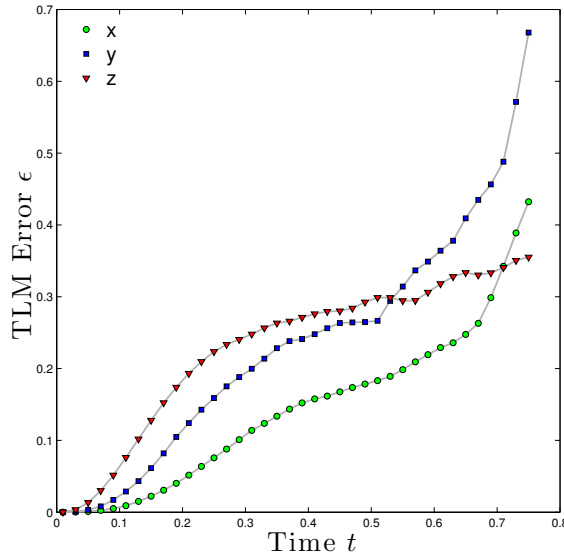


Figure 2.3: The future error predicted by the TLM is compared to the error growth in Lorenz 63 system for an initial perturbation with standard deviation of 0.1, averaged over 1000 TLM integrations. The ϵ is not the error predicted by the TLM, but rather the error of the TLM in predicting the error growth.

2.4 Ensemble Kalman Filter

The computational cost of the EKF is mitigated through the approximation of the error covariance matrix $\mathbf{P}_f$ from the model itself, without the use of a TLM.

One such approach is the use of a forecast ensemble, where a collection of models (ensemble members) are initialized from the prior analysis $\mathbf{x}_a$ with random error. The spread of these models with random errors is then used to estimate the error covariance. In the limit of infinite ensemble size, the EnKF is equivalent to the EKF (Evensen 2003). There are two prevailing methodologies for maintaining independent ensemble members: adding errors to the observation used by each forecast and assimilating each ensemble individually or distributing ensemble initial conditions from the analysis prediction. In the prior, the random error added to the observation can be truly random (Harris et al. 2011) or be drawn from a distribution related to the analysis covariance, which I chose for this type of filter. The latter method of distributing ensemble members around the analysis forecast, with initial conditions perturbed as (for example) the square root of the ensemble has advantages compared to perturbed observations (Evensen 2003). It has been shown that the perturbed observation approach introduces additional sampling error that reduces the analysis covariance accuracy and increases the probability of underestimating analysis error covariance (Hamill et al. 2001).

With a finite ensemble size this method is only an approximation and therefore in practice it often fails to capture the full spread of error. To better capture the model variance, additive and multiplicative inflation factors are used to obtain a good estimate of the error covariance matrix. The specific factors used are tuned through an exhaustive search, and included in the next section. The spread of ensemble members in the x_1 variable of the Lorenz model, as distance from the analysis, can be seen in Figure 2.4

The only difference between this approach and the EKF, in general, is that the forecast error covariance $\mathbf{P}^f$ is computed from the ensemble members, without the need for a tangent linear model:

$$\mathbf{P}^f \approx \frac{1}{K-2} \sum_{k \neq l} \left(\mathbf{x}_k^f - \bar{\mathbf{x}}^f \right) \left(\mathbf{x}_l^f - \bar{\mathbf{x}}^f \right)^T.$$

In computing the error covariance $\mathbf{P}_f$ from the ensemble, we wish to add up the error covariance of each forecast with respect to the mean forecast. But this would underestimate the error covariance since the forecast we're comparing was used in the ensemble average (to obtain the mean forecast). So here to

CHAPTER 2. DA

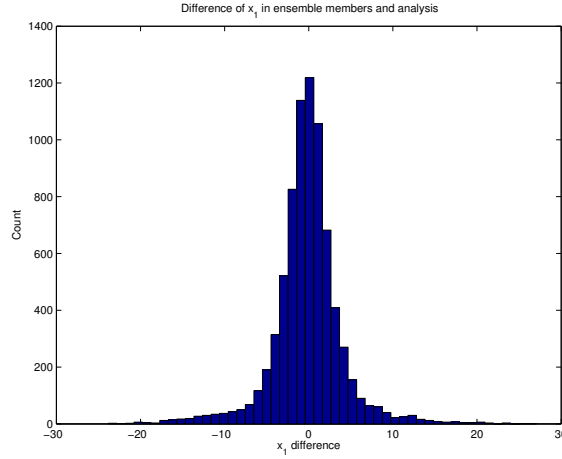


Figure 2.4: The difference of ensemble forecasts from the analysis is reported for 760 assimilation windows in one model run of length 200, with 10 ensemble members and an assimilation window of length 0.261. This has the same shape of as the difference between ensemble forecasts and the mean of the forecasts (I have this as well). This spread of ensemble forecasts is what allows us to estimate the error covariance of the forecast model, and appears fall from a normal distribution.

computing the error covariance matrix for each forecast, that forecast itself is excluded from the ensemble average forecast.

We can see the classic spaghetti of the ensemble with this filter implemented on Lorenz 63 in Figure 2.5.

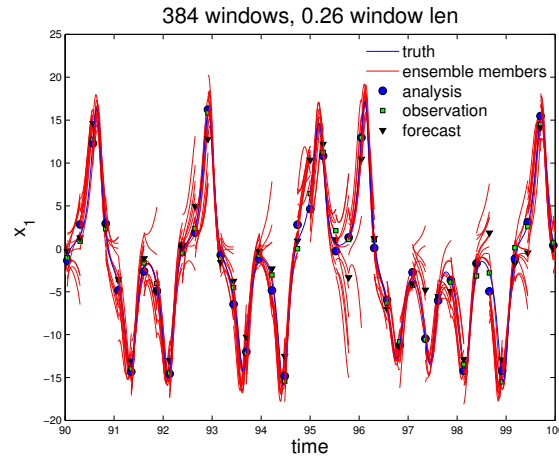


Figure 2.5: A sample timeseries of the ensembles used in the EnKF. In all tests, as seen here, 10 ensemble members are used.

The forecast from an ensemble is the average of the ensemble forecasts, and an explanation for this choice is substantiated by Burgers (Burgers et al. 1998). The general EnKF which I use is most similar to that of Burgers. Many algorithms based on the EnKF have been proposed and include the Ensemble Transform Kalman Filter (ETKF) (Ott et al. 2004), Ensemble Analysis Filter (EAF) (Anderson 2001), Ensemble Square Root Filter (EnSRF) (Tippett et al. 2003), Local Ensemble Kalman Filter (LEKF) (Ott et al. 2004), and finally the Local Ensemble Transform Kalman Filter (LETKF) (Hunt et al. 2007). I explore some of these in the following sections. A comprehensive overview through 2003 is provided by Evensen (Evensen 2003). Next, I present the results of a MATLAB implementation of the EKF and basic EnKF.

2.4.1 Performace of EnKF, EKF on Lorenz 63

With the EnKF and the EKF algorithms coded as above, I present some basic results on their performance. First, we see that using a data assimilation algorithm is highly advantageous in general (Figure 2.6). Of course we could think to use the observations directly when DA is not used, but since we only observing the x_1 variable (and in real situations measurements are not so direct), this is not prudent.

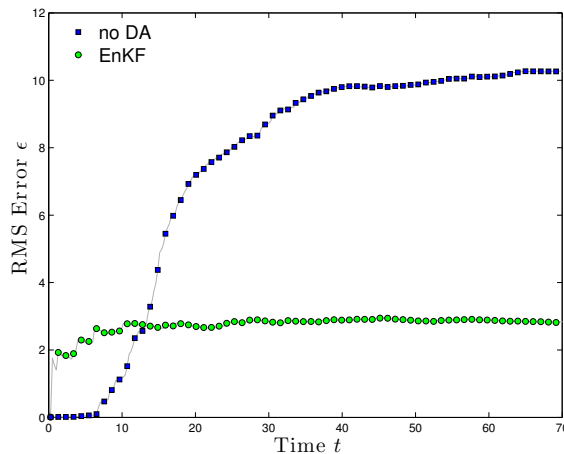


Figure 2.6: For near-perfect initial knowledge of the atmospheric state, the RMS error of a prediction using DA is compared against a prediction that does not. Initial error is normal with mean 0 and variance 0.01, and the RMS errors reported are averages of 100 runs. Observational noise is normally distributed with mean 0 and variance 0.5, and an assimilation window length of 0.261 is used (approximately 5% of climatological variation). As would be expected, in the long run DA greatly improves forecast skill, as forecasts without DA saturate to climatological error. The initial period where a forecast exceeds the skill of DA is potentially due to the spin-up time required by the filter to dial in the analysis error covariance.

Next we test the performance of each filter against increasing assimilation window length. As we would expect, longer windows make forecasting more challenging for both algorithms. The ensemble filter begins to out-perform the extended kalman filter for longer window lengths because the ensemble more accurately captures the nonlinearity of the model.

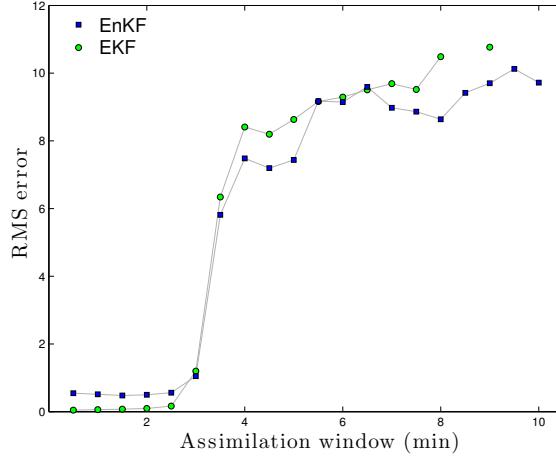


Figure 2.7: The RMS error (unscaled by climatology) is reported for our EKF and EnKF filters, measured as the difference between forecast and truth at the end of an assimilation window for the latter 2500 assimilation windows in a 3000 assimilation window model run. Error is measured in the only observed variable, x_1 . Increasing the assimilation window led to an decrease in predictive skill, as expected. Additive and multiplicative covariance inflation is the same as Harris et. al.

Both filters are sensitive to the size of the covariance matrix, and in some cases to maintain the numerical stability of the assimilation it is necessary to use additive and/or multiplicative covariance inflation. In both the EKF and the EnKF mutliplicative error is performed with inflection factor Δ by

$$\mathbf{K} = (1 + \Delta)\mathbf{K}. \quad (2.42)$$

. Additive inflation is performed for the EKF as in (Yang et al. 2006) with inflection factor μ , where uniform random numbers in $[0, \mu]$, here ν representing a vector of length n of such numbers:

$$\mathbf{K} = \mathbf{K} + \text{diag}(\nu). \quad (2.43)$$

For the EnKF we perform additive covariance inflation to the analysis state itself for each ensemble:

$$\mathbf{x} = \mathbf{x} + \nu. \quad (2.44)$$

The result of search over these parameters for the optimal values to minimize the analysis RMS error differs for each window length, so here we present the result for a window of length 210 seconds on the EKF. In Figure 2.8 we see that there is a clear region of parameter space for which the filter performs with lower RMS, although this region appears to extend into values of $\Delta > 3$.

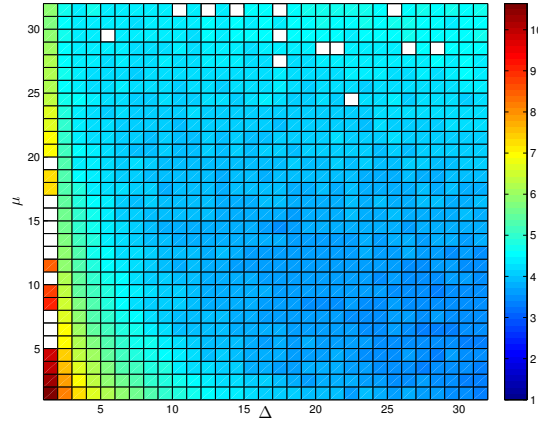


Figure 2.8: The RMS error averaged over 100 model runs of length 1000 windows is reported for the EKF for varying additive and multiplicative inflation factors Δ and μ . Each of the 100 model runs starts with a random IC, and the analysis forecast starts randomly. The filter performance RMS is computed as the RMS value of the difference between forecast and truth at the assimilation window for the latter 500 windows, allowing a spin-up of 500 windows.

2.4.2 Ensemble Transform Kalman Filter (ETKF)

The ETKF introduced by Bishop is one type of square root filter, and I present it here to provide background for the formulation of the LETKF (Bishop et al. 2001). For a square root filter in general, we begin by writing the covariance matrices as the product of their matrix square roots. Because $\mathbf{P}_a, \mathbf{P}_f$ are symmetric positive-definite (by definition) we can write

$$\mathbf{P}_a = \mathbf{Z}_a \mathbf{Z}_a^T, \quad \mathbf{P}_f = \mathbf{Z}_f \mathbf{Z}_f^T \quad (2.45)$$

CHAPTER 2. DA

for $\mathbf{Z}_a, \mathbf{Z}_f$ the matrix square roots of $\mathbf{P}_a, \mathbf{P}_f$ respectively. We are not concerned that this decomposition is not unique, and note that $\mathbf{Z}$ must have the same rank as $\mathbf{P}$ which will prove computationally advantageous. The power of the SRF is now seen as we represent the columns of the matrix $\mathbf{Z}_f$ as the difference from the ensemble members from the ensemble mean, to avoid forming the full forecast covariance matrix $\mathbf{P}_f$. The ensemble members are updated by applying the model M to the states $\mathbf{Z}_f$ such that an update is performed by

$$\mathbf{Z}_f = M\mathbf{Z}_a. \quad (2.46)$$

Per (?), the solution for $\mathbf{Z}_a$ can be solved by perturbed observations

$$\mathbf{Z}_a = (\mathbf{I} - \mathbf{KH})\mathbf{Z}_f + \mathbf{KW} \quad (2.47)$$

where $\mathbf{W}$ is the observation perturbation of Gaussian random noise This gives the analysis with the expected statistics

$$\langle \mathbf{Z}_a \mathbf{Z}_a^T \rangle = (\mathbf{I} - \mathbf{KH})\mathbf{P}_f(\mathbf{I} - \mathbf{KH})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T \quad (2.48)$$

$$= \mathbf{P}_a \quad (2.49)$$

but as mentioned earlier, this has been shown to decrease the resulting covariance accuracy.

In the SRF approach, the ‘‘Potter Method’’ provides a deterministic update by rewriting

$$\mathbf{P}_a = \mathbf{Z}_a \mathbf{Z}_a^T = (\mathbf{I} - \mathbf{P}_f \mathbf{H}^T (\mathbf{R} + \mathbf{H} \mathbf{P}_f \mathbf{H}^T)^{-1} \mathbf{H}) \mathbf{P}_f \quad (2.50)$$

$$= \mathbf{Z}_f (\mathbf{I} - \mathbf{Z}_f \mathbf{H}^T (\mathbf{H} \mathbf{Z}_f \mathbf{Z}_f^T \mathbf{H}^T + \mathbf{R})^{-1} \mathbf{H} \mathbf{Z}_f) \mathbf{Z}_f^T \quad (2.51)$$

$$= \mathbf{Z}_f (\mathbf{I} - \mathbf{V} \mathbf{D}^{-1} \mathbf{Z}_f^T) \mathbf{Z}_f^T. \quad (2.52)$$

We have defined $\mathbf{V} \equiv (\mathbf{H} \mathbf{Z}_f)^T$ and $\mathbf{D} \equiv \mathbf{V}^T \mathbf{V} + \mathbf{R}$. Now for the ETFK step, we use the Sherman-Morrison-Woodbury identity to show

$$\mathbf{I} - \mathbf{Z} \mathbf{D}^{-1} \mathbf{Z}^T = (\mathbf{I} + \mathbf{Z}_f \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} \mathbf{Z}_f)^{-1}. \quad (2.53)$$

The matrix on the right hand side is practical to compute because it is of dimension of the rank of the ensemble, assuming that the matrix inverse $\mathbf{R}^{-1}$ is available. This is the case in our experiment, as the

CHAPTER 2. DA

observation are uncorrelated. The analysis update is thus

$$\mathbf{Z}_a = \mathbf{Z}_f \mathbf{C} (\mathbf{T} + \mathbf{I})^{-1/2}, \quad (2.54)$$

where $\mathbf{C} \mathbf{T} \mathbf{C}^T$ is the eigenvalue decomposition of $\mathbf{Z}_f^T \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} \mathbf{Z}_f$. From (?) we note that the matrix $\mathbf{C}$ of orthonormal vectors is not unique.

Then the analysis perturbation is calculated from

$$\mathbf{Z}_a = \mathbf{Z}_f \mathbf{X} \mathbf{U}, \quad (2.55)$$

where $\mathbf{X} \mathbf{X}^T = (\mathbf{I} - \mathbf{Z} \mathbf{D}^{-1} \mathbf{V}^T)$ and $\mathbf{U}$ is an arbitrary orthogonal matrix.

In this filter, the analysis perturbations are assumed to be equal to the background perturbations postmultiplied by a transformation matrix $\mathbf{T}$ so that the analysis error covariance satisfies

$$\mathbf{P}_a = (1 - \mathbf{K} \mathbf{H}) \mathbf{P}_f.$$

The analysis covariance is also written

$$\mathbf{P}_f = \frac{1}{P-1} (\mathbf{X}_a (\mathbf{X}_a)^T = \mathbf{X}^b \hat{\mathbf{A}} (\mathbf{X}^b)^T$$

where $\hat{\mathbf{A}} = [(P-1)\mathbf{I} + (\mathbf{H} \mathbf{X}^b)^T \mathbf{R}^{-1} (\mathbf{H} \mathbf{X}^b)]^{-1}$. The analysis perturbations are $\mathbf{X}^a = \mathbf{X}^b \mathbf{T}$ where $\mathbf{T} = [(P-1)\hat{\mathbf{A}}]^{1/2}$.

To summarize, the steps for the ETKF are to (1) form $\mathbf{Z}_f^T \mathbf{H}^T \mathbf{R}^{-1} \mathbf{H} \mathbf{Z}_f$, assuming $\mathbf{R}^{-1}$ is easy. Then (2) compute the eigenvalue decomposition of this, and apply it $\mathbf{Z}_f$.

2.4.3 Local Ensemble Kalman Filter (LEKF)

The LEKF implements a strategy that becomes very important for large simulations: localization. Namely, the analysis is computed for each gridpoint using only local observations, without the need to build matrices that represent the entire analysis space. This localization removes long-distance correlations from $\mathbf{B}$ and allows

CHAPTER 2. DA

greater flexibility in the global analysis by allowing different linear combinations of ensemble members at different spatial locations (Kalnay et al. 2007).

Difficulties are presented however, in the form of computational grid dependence for the localization procedure. The toroidal shape of our experiment, when discretized by OpenFOAM's mesher and grid numbering optimized for computational bandwidth for solving, does not require that adjacent grid points have adjacent indices. In fact, they are decidedly non-adjacent for certain meshes, with every other index corresponding to opposing sides of the loops. We overcome this difficulty by placing a more stringent requirement on the mesh of localization of indexing, and extracting the adjacency matrix for use in filtering.

The general formulation of the LEKF by Ott (Ott et al. 2004) goes as follows, directly from his paper:

1. Globally advance each ensemble member to the next analysis timestep. Steps 2-5 are performed for each grid point
2. Create local vectors from each ensemble member
3. Project that point's local vectors from each ensemble member into a low dimensional subspace as represented by perturbations from the mean
4. Perform the data assimilation step to obtain a local analysis mean and covariance
5. Generate local analysis ensemble of states
6. Form a new global analysis ensemble from all of the local analyses
7. Wash, rinse, and repeat.

Figure 2.9 depicts this cycle in greater detail, with similar notations to that used thus far (from (Ott et al. 2004)).

There is not much difference than from what one would build naturally to perform the EnKF with localization in Ott's work (Ott et al. 2004), with the exception of the reconstruction of the global analysis from local analyses. Since each of the assimilations were performed locally, we cannot be sure that there is any smoothness, or physical balance, between adjacent points. To ensure a unique solution of local analysis perturbations that has some respect of this balance, utilization of the prior background forecasts as reasonably accurate is used to constrain the construction of local analysis perturbations to minimize the difference from the background.

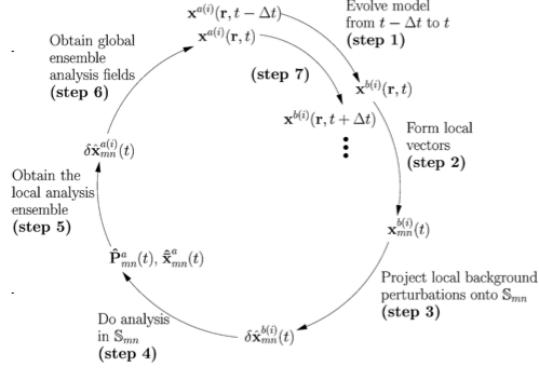


Figure 2.9: The algorithm for the LEKF as described by Ott. Building on previous work, the algorithm is as one would expect.

2.4.4 Local Ensemble Transform Kalman Filter (LETKF)

Proposed by Hunt in 2007 with the stated objective of computational efficiency, the LETKF is named from its most similar algorithms from which it draws (Hunt et al. 2007). With the formulation of the LEKF and the ETKF there is not much new to be said here, but rather a synthesis of the advantages of both of these approaches. This is the method that was sufficiently efficient for implementation on the full OpenFOAM CFD model of 216,000 model variables, and so I present it in more detail and follow the notation of (Hunt et al. 2007).

As in the LEKF we explicitly perform the analysis for each grid point of the model. The choice of observations to use for each grid point can be selected a priori, and tuned adaptively. The simplest choice is to define a radius of influence for each point, and use observations within that. In addition to this simple scheme, Hunt proposes weighing observations by their proximity to smoothly transition across them. The goal is to avoid sharp boundaries between neighboring points.

Regarding the LETKF analysis problem as the construction of an analysis ensemble from the background ensemble, where we specify that the mean of the analysis ensemble minimizes a cost function, we state the equations in terms of a cost function. The LETKF is thus

$$\{\mathbf{x}_{b(i)} : i = 1..k\} \xrightarrow{\text{LETKF}} \{\mathbf{x}_{a(i)} : i = 1..k\} \quad (2.56)$$

CHAPTER 2. DA

where

$$J(\mathbf{x}) = (\mathbf{x} - \bar{\mathbf{x}}_b)^T (\mathbf{P}_b)^T (\mathbf{x} - \bar{\mathbf{x}}_b) + (\mathbf{y}_o - H(\mathbf{x}))^T \mathbf{R}^{-1} (\mathbf{y}_o - H(\mathbf{x})).$$

Since we have defined $\mathbf{P}_b = (1/(k-1))\mathbf{X}_b\mathbf{X}_b^T$ for the $m \times k$ matrix $\mathbf{X}_b = (\mathbf{x}_{b(1)}, \dots, \mathbf{x}_{b(k)})$, the rank of $\mathbf{P}_b$ is at most $k-1$, and the inverse of $\mathbf{P}$ is not defined. To ameliorate this, we consider the analysis computation to take place in the subspace S spanned by the columns of $\mathbf{X}_b$ (the ensemble states), and regard $\mathbf{X}_b$ as a linear transformation from a k -dimensional space $\tilde{S}$ into S . Letting $\mathbf{w}$ be a vector in $\tilde{S}$, then $\mathbf{X}_b\mathbf{w}$ belongs to S and $\mathbf{x} = \bar{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}$. We can therefore write the cost function as

$$\tilde{J} = \frac{1}{k-1} \mathbf{w}^T \mathbf{w} + (\mathbf{y}_o - H(\bar{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}))^T \times \mathbf{R}^{-1} (\mathbf{y}_o - H(\bar{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}))$$

and it has been shown that this is an equivalent formulation by (Hunt et al. 2007).

Finally, we linearize H by applying it to each $\mathbf{x}_{b(i)}$ and interpolating. This is accomplished by defining

$$\mathbf{y}_{b(i)} = H(\mathbf{x}_{b(i)}) \tag{2.57}$$

such that

$$H(\bar{\mathbf{x}}_b + \mathbf{X}_b\mathbf{w}) = \bar{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w} \tag{2.58}$$

where $\mathbf{Y}$ is the $l \times k$ matrix whose columns are $\mathbf{y}_{b(i)} - \bar{\mathbf{y}}_b$ for $\bar{\mathbf{y}}_b$ the average of the $\mathbf{y}_{b(i)}$ over i . Now we rewrite the cost function for the linearize observation operator as

$$\tilde{J}^* = (k-1) \mathbf{w}^T \mathbf{w} + (\mathbf{y}_o - \bar{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w})^T \times \mathbf{R}^{-1} (\mathbf{y}_o - \bar{\mathbf{y}}_b - \mathbf{Y}_b\mathbf{w})$$

CHAPTER 2. DA

This cost function minimum is the form of the Kalman Filter as I have written it before:

$$\begin{aligned}\bar{\mathbf{w}}_a &= \tilde{\mathbf{P}}_a (\mathbf{Y}_b)^T \mathbf{R}^{-1} (\mathbf{y}_o - \bar{\mathbf{y}}_b), \\ \tilde{P}_a &= ((k-1)\mathbf{I} + (\mathbf{Y}_b)^T \mathbf{R}^{-1} \mathbf{Y}_b)^{-1}.\end{aligned}$$

In model space these are

$$\begin{aligned}\tilde{\mathbf{x}}_a &= \bar{\mathbf{x}}_b + \mathbf{X}_b \bar{\mathbf{w}}_a \\ \mathbf{P}_a &= \mathbf{X}_b \tilde{\mathbf{P}}_a (\mathbf{X}_b)^T.\end{aligned}$$

We move directly to a detailed description of the computational implementation of this filter. Starting with a collection of background forecast vectors $\{\mathbf{x}_{b(i)} : i = 1..k\}$, we perform steps 1 and 2 in a global variable space, then steps 3-8 for each grid point:

1. apply H to $\mathbf{x}_{b(i)}$ to form $\mathbf{y}_{b(i)}$, average the $\mathbf{y}_b$ for $\bar{\mathbf{y}}_b$, and form $\mathbf{Y}_b$.
2. similarly form $\mathbf{X}_b$. (now for each grid point)
3. form the local vectors
4. compute $\mathbf{C} = (\mathbf{Y}_b)^T \mathbf{R}^{-1}$ (perhaps by solving $\mathbf{R}\mathbf{C}^T = \mathbf{Y}_b$)
5. compute $\tilde{\mathbf{P}}_a = ((k-1)\mathbf{I}/\rho + \mathbf{C}\mathbf{Y}_b)^{-1}$ where $\rho > 1$ is a tunable covariance inflation factor
6. compute $\mathbf{W}_a = \left((k-1)\tilde{\mathbf{P}}_a\right)^{1/2}$
7. compute $\bar{\mathbf{w}}_a = \tilde{\mathbf{P}}_a \mathbf{C} (\mathbf{y}_o - \bar{\mathbf{y}}_b)$ and add it to the column of $\mathbf{W}_a$
8. multiply $\mathbf{X}_b$ by each $\mathbf{w}_{a(i)}$ and add $\tilde{\mathbf{x}}_b$ to get $\{\mathbf{x}_{a(i)} : i = 1..k\}$ to complete each grid point
9. combine all of the local analysis into the global analysis

The greatest implementation difficulty in the OpenFOAM model comes back to the spatial discretization and defining the local vectors for each grid point.

Four Dimensional Local Ensemble Transform Kalman Filter (4D-LETKF)

Extending the LETKF to incorporate observations from within the assimilation time window also become useful with high-frequency temperature sensors. This is implemented without great difficulty on top of the LETKF, by projecting observations as they are made into the space $\tilde{S}$ with the observation time-specific matrix H_τ and concatenating each successive observation onto $\mathbf{y}_o$. Then with the background forecast at the time τ , we compute $(\bar{\mathbf{y}}_b)_\tau$ and $(\mathbf{Y}_b)_\tau$, concatenating onto $\bar{\mathbf{y}}_b$ and $\mathbf{Y}_b$, respectively.

Table 2.1: Summary of Lorenz '63 parameters

Parameter	Value	Interpretation, if any
σ	10	
b	8/3	
r	28	

The canonical choice of $\sigma = 10$, $b = 8/3$ and $r = 28$ is used for all tests here, which produce the well known butterfly attractor (Figure 2.1).

2.5 4D-Var

The formulation of 4D-Var aims to incorporate observations at the time they are observed. This inclusion of observations has resulted in significant improvements in forecast skill, cite. The next generation supercomputers that will be used in numerical weather prediction will allow for increased model resolution, but the proper scaling of this computation with the number of processors. Main issues with the 4D-Var approach include attempting to parallelize what is otherwise a sequential algorithm and the computational difficulty of the strong-constraint formulation with many TLM integrations, see work done by (Fisher and Andersson 2001).

The formulation of 4D-Var is not so different from 3D-Var, with the inclusion of a TLM and adjoint model to assimilation observations made within the assimilation window. At the ECMWF, their 4D-Var is run on a long-window cycle (24 and 48 hours) so that many observations are made within the time window. Generally, there are two main 4D-Var formulations in use: strong-constraint and weak-constraint. The strong-constraint formulation uses the forecast model as a strong constraint, and is therefore significantly easier

CHAPTER 2. DA

to solve. From (Kalnay et al. 2007) we have the standard weak-constraint formulation with superscript denoting the timestep within the assimilation window (say there are $0, \dots, N$ observation time steps within the assimilation window):

$$J(\mathbf{x}^0) = \frac{1}{2}(\mathbf{x}^0 - \mathbf{x}_b^0)^T \mathbf{B}^{-1}(\mathbf{x}^0 - \mathbf{x}_b^0) + \frac{1}{2} \sum_{i=0}^N (H^i(\mathbf{x}^i) - \mathbf{y}^i)^T (\mathbf{R}^i)^{-1} (H^i(\mathbf{x}^i) - \mathbf{y}^i).$$

The weak-constraint formulation has only recently been implemented operationally. This is (Kalnay et al. 2007):

$$J(\mathbf{x}^0, \beta) = \frac{1}{2}(\mathbf{x}^0 - \mathbf{x}_b^0)^T \mathbf{B}^{-1}(\mathbf{x}^0 - \mathbf{x}_b^0) + \frac{1}{2} \sum_{i=1}^N (H^i(\mathbf{x}^i + \beta) - \mathbf{y}^i)^T (\mathbf{R}^i)^{-1} (H^i(\mathbf{x}^i + \beta) - \mathbf{y}^i) + \frac{1}{2} \beta^T \mathbf{Q}^{-1} \beta = J_b + J_o + J_\beta.$$

Future directions of operational data assimilation combine what is best about 4D-Var and ensemble-based algorithms, and these methods are referred to as ensemble-variational hybrids. We have already discussed a few such related approaches: the use of the Kalman gain matrix $\mathbf{K}$ used within the 3D-Var cost function, the “NCEP” method for 3D-Var, and inflating the ETKF covariance with the 3D-Var static (climatological) covariance $\mathbf{B}$.

2.5.1 Ensemble-variational hybrids

In ensemble-variational hybrid approaches benefits of flow dependent covariance estimates obtained from an ensemble method, and the computational efficiency of iteratively solving a cost function to obtain the analysis, are combined. An ensemble of forecasts is used to update the static error covariance $\mathbf{B}$ in the formulation of 4D-Var (this could be done in 3D-Var as well) with some weighting from the climatological covariance $\mathbf{B}$ and the ensemble covariance. This amounts to a formulation of

$$\mathbf{B} = \alpha \mathbf{P}_f + (1 - \alpha) \mathbf{B} \tag{2.59}$$

CHAPTER 2. DA

for some α between 0 and 1. Combining the static error covariance $\mathbf{B}$ and the flow dependent error covariance $\mathbf{P}_f$ is an active area of research (Clayton et al. 2012).

Chapter 3

Computational Fluid Dynamics

3.1 Derivation of Governing Equations

Closely following the original derivation of Stokes (Stokes 1846), I present a derivation of the main equations governing the flow of water at temperatures near 300K and Rayleigh number within reason. These three equations are given by

$$\begin{aligned}\frac{\partial \rho}{\partial t} + \frac{\partial}{\partial x_j} [\rho u_j] &= 0 \\ \frac{\partial}{\partial t} (\rho u_i) + \frac{\partial}{\partial x_j} [\rho u_i u_j + p \delta_{ij} - \tau_{ji}] &= 0, \quad i = 1, 2, 3 \\ \frac{\partial}{\partial t} (\rho e_0) + \frac{\partial}{\partial x_j} [\rho u_j e_0 + u_j p + q_j - u_i \tau_{ij}] &= 0\end{aligned}$$

3.1.1 Continuity Equation

Considering a finite volume element with side lengths Δx , Δy and Δz . The amount of mass that enters any given face on this volume is a function of the fluid density ρ , the velocity tangent to this face, and the area of the face. For the a side with edges specified by both Δx and Δy , let the velocity tangent to this face be u and

CHAPTER 3. CFD

this mass flow in this side is equal to

$$\rho u \Delta x \Delta y.$$

Assuming the u is positive in this direction, the mass flux out of the opposite side needs to reflect possible changes in velocity u and density ρ through the volume and can be written

$$-(\rho + \Delta\rho)(u + \Delta u) \Delta x \Delta y.$$

Similarly for the other two directions tangent to our volume, we assign velocities v and w with incoming mass

$$\rho v \Delta x \Delta z$$

$$\rho w \Delta y \Delta z$$

and outgoing mass

$$-(\rho + \Delta\rho)(v + \Delta v) \Delta x \Delta z$$

$$-(\rho + \Delta\rho)(w + \Delta w) \Delta y \Delta z.$$

The rate of mass accumulation in the volume,

$$\Delta\rho(\Delta x \Delta y \Delta z) / \Delta t,$$

must be equal to the sum of the mass entering and leaving the volume (given by the six equations above).

Equating these, with a little cancellation and dividing by $(\Delta x \Delta y \Delta z)$ we are left with

$$\frac{\Delta\rho}{\Delta t} + \frac{\Delta(\rho u)}{\Delta x} + \frac{\Delta(\rho v)}{\Delta y} + \frac{\Delta(\rho w)}{\Delta z} = 0. \quad (3.1)$$

As $\Delta t \rightarrow 0$, this can be written in terms of the partial derivatives

$$\frac{\partial\rho}{\partial t} + \frac{\partial(\rho u)}{\partial x} + \frac{\partial(\rho v)}{\partial y} + \frac{\partial(\rho w)}{\partial z} = 0. \quad (3.2)$$

CHAPTER 3. CFD

For an incompressible fluid, we have that $\partial\rho/\partial\{t, x, y, z\} = 0$ such the continuity equation becomes

$$\frac{\partial u}{\partial x} + \frac{v}{\partial y} + \frac{\partial w}{\partial z} = 0. \quad (3.3)$$

3.1.2 Momentum Equation

3.1.3 Energy Equation

3.2 Discretization and Solving Algorithms

A spatial and temporal discretization of the governing equations is necessary for their solution numerically.

Specifically, a combined SIMPLE/PISO algorithm is used. The PISO (Pressure-Implicit with Splitting of Operators) algorithm derives from the work of (Issa 1986), and is complementary to the SIMPLE (Semi-Implicit Method for Pressure-Linked Equations) (Patankar and Spalding 1972) iterative method.

Simple algorithm: http://www.cfd-online.com/Wiki/SIMPLE_algorithm

The proper inclusion of turbulent effects with calculated Raleigh number are important, and are modeled using LES (Large Eddy Simulation).

3.3 Solving with OpenFOAM

Using the aforementioned numerical schemes, I perform a computational fluid dynamic simulation of the entire thermosyphon using the open-source software OpenFOAM. First, we begin by creating 2 and 3-dimensional meshes of the thermosyphon natively in OpenFOAM's blockMesh utility. To reduce computational bandwidth, it was important to first create the mesh and then renumber using the renumberMesh utility. The 2D mesh contains 36000 points, and is imposed with fixed flux boundary conditions on the top and bottom of the loop. This fixed flux boundary condition is implemented through the externalWallHeatFluxTemperature library, with some difficulty. The 3D mesh contains over 1M points, and we have chosen to explore the 2D model for now.

Links for OpenFOAM code documentation:

<http://www.openfoam.org/docs/user/compiling-applications.php>

<http://www.openfoam.org/docs/cpp/>

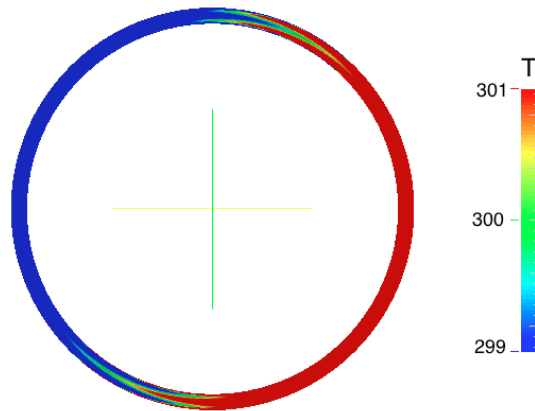


Figure 3.1: A screenshot of the loop.

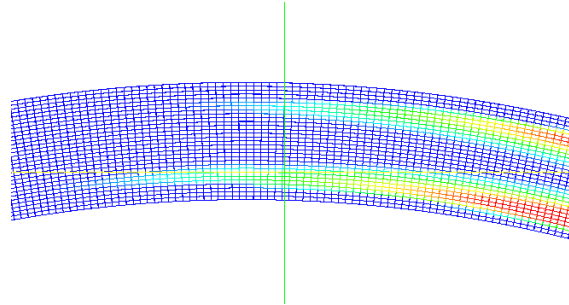


Figure 3.2: A snapshot of the mesh used for CFD simulations.

Here is a picture of the thermosyphon colored by temperature.

And here is the 2D mesh:

The solver used is `buoyantBoussinesqPimpleFoam`, which I'll now explain.

The Boussinesq approximation makes the assumption that density changes due to temperature differences are sufficiently small so that they can be neglected without affecting the flow, except when multiplied by gravity. The non-dimensionalized density difference $\Delta\rho/\rho$ depends on the value of heat flux into the loop, but remains small even for large values of flux. (derive an equation for this ratio in terms of flux q'').

Chapter 4

Thermal Convection Loop Experiment

The thermosyphon, a type of natural convection loop or non-mechanical heat pump, can be likened to a toy model of climate (Harris et al. 2011).

4.1 Thermal Convection Loops

Under certain conditions, thermal convection loops can operate as non-mechanical heat pumps, making them useful in many applications.

4.2 Ehrhard-Müller Equations

Following the derivation by Harris (Harris et al. 2011), itself a representation of the derivation of Gorman (Gorman et al. 1986) and namesakes Ehrhard and Müller (Ehrhard and Müller 1990).

4.3 Detailed Experimental Setup

Originally built (and subsequently melted) by Dr. Chris Danforth, the University of Vermont's current experimental thermosyphons have made considerable progress. Operated by Dave Hammond, a Scientific Electronics Technician and master parameter tuner, the experimental thermosyphons access the chaotic regime of state space found in the principled governing equations.

CHAPTER 4. THERMOSYPHON



Figure 4.1: Professor Chris Danforth with the aluminum rig.

Chapter 5

Results

5.1 Prediction Skill for Synthetic Data

I first present the results pertaining to the accuracy of forecasts for synthetic data.

5.1.1 Sampling frequency, distribution

Increasing sampling frequency led to improved forecast accuracy, up to a finite cutoff where the error covariance inflation requirement becomes unstable.

Observational network density also improves forecast accuracy, as would be expected, up to limit of the computational memory available for computing large non-sparse covariance matrices. For example the full system, with 216,000 variables, produces an error covariance matrix of 347GB in double precision. With the expected continuation of Moore's Law, however, this simulation may be possible in a few as 5 years (assuming a doubling of memory every 18 months, starting now with my 8GB laptop) on a laptop computer.

5.1.2 Random error vs PMD perturbation scheme

The computational implementation of the NS equations is dispersive, meaning that small perturbations are quickly resolved.

In other words, we beat the hell of out random error.

CHAPTER 5.

5.1.3 Maintaining hydrostatic balance

The importance of maintaining balance in the model variables, originally dooming the efforts of Richardson, was taken into consideration.

5.2 Real-time Flow Reversal Prediction Skill

Now the real business: running CFD in real time and assimilating observations from temperature probes.

Chapter 6

Conclusion

6.1 Main Result

Here is a different citation (Bongard and Paul 2000).

6.1.1 More Details

And one more (Auerbach and Bongard 2010).

BIBLIOGRAPHY

Bibliography

- Achtelik, M., T. Zhang, K. Kuhnlenz, and M. Buss (2009). Visual tracking and control of a quadcopter using a stereo camera system and inertial sensors. In *Mechatronics and Automation, 2009. ICMA 2009. International Conference on*, pp. 2863–2869. IEEE.
- Anderson, M. J. (2001). A new method for non-parametric multivariate analysis of variance. *Austral Ecology* 26(1), 32–46.
- Asur, S. and B. A. Huberman (2010). Predicting the future with social media. In *Web Intelligence and Intelligent Agent Technology (WI-IAT), 2010 IEEE/WIC/ACM International Conference on*, Volume 1, pp. 492–499. IEEE.
- Auerbach, J. E. and J. C. Bongard (2010). Evolving CPPNs to grow three-dimensional physical structures. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2010)*, New York, NY, pp. 627–634. ACM.
- Beine, B., V. Kaminski, and W. Von Lensa (1992). Integrated design of prestressed cast-iron pressure vessel and passive heat removal system for the reactor cell of a 200 mwth modular reactor. *Nuclear engineering and design* 136(1), 135–141.
- Beitelmal, M. H. and C. D. Patel (2002). Two-phase loop: Compact thermosyphon. *Approved for External Publication*.
- Belessiotis, V. and E. Mathioulakis (2002). Analytical approach of thermosyphon solar domestic hot water system performance. *Solar Energy* 72(4), 307–315.
- Bishop, C. H., B. J. Etherton, and S. J. Majumdar (2001). Adaptive sampling with the ensemble transform kalman filter. part i: Theoretical aspects. *Monthly weather review* 129(3), 420–436.
- Bollen, J., H. Mao, and X. Zeng (2011). Twitter mood predicts the stock market. *Journal of Computational Science* 2(1), 1–8.
- Bongard, J. C. and C. Paul (2000). Investigating morphological symmetry and locomotive efficiency using virtual embodied evolution. In *From Animals to Animats: The Sixth International Conference on the Simulation of Adaptive Behaviour*, pp. 420–429. MIT Press.
- Burgers, G., P. Jan van Leeuwen, and G. Evensen (1998). Analysis scheme in the ensemble kalman filter. *Monthly weather review* 126(6), 1719–1724.
- Burroughs, E., E. Coutsias, and L. Romero (2005). A reduced-order partial differential equation model for the flow in a thermosyphon. *Journal of Fluid Mechanics* 543, 203–238.
- Charney, J. (1951). Dynamic forecasting by numerical process. *Compendium of Meteorology, American Meteorological Society, Boston*, 470–482.
- Cherry, B., Z. Dan, and A. Lustgarten (2006). Next stop, lhasa. *Fortune* 153, 154.
- Clayton, A., A. Lorenc, and D. Barker (2012). Operational implementation of a hybrid ensemble/4d-var global data assimilation system at the met office. *Quarterly Journal of the Royal Meteorological Society*.
- Creveling, H., D. PAZ, J. Baladi, and R. Schoenhals (1975). Stability characteristics of a single-phase free convection loop. *Journal of Fluid Mechanics* 67(part 1), 65–84.
- Dee, D. P. and A. M. Da Silva (1998). Data assimilation in the presence of forecast bias. *Quarterly Journal of the Royal Meteorological Society* 124(545), 269–295.
- Desrayaud, G., A. Fichera, and M. Marcoux (2006). Numerical investigation of natural circulation in a 2d-annular closed-loop thermosyphon. *International journal of heat and fluid flow* 27(1), 154–166.
- Detman, R. F. (1968, July 16). Thermos iphon depp pool reactor. US Patent 3,393,127.

BIBLIOGRAPHY

- Edwards, A. W. F. (1984). Likelihood: An account of the statistical concept of likelihood and its application to scientific inferences.
- Ehrhard, P. and U. Müller (1990). Dynamical behaviour of natural convection in a single-phase loop. *Journal of Fluid mechanics* 217, 487–518.
- Evensen, G. (2003). The ensemble kalman filter: Theoretical formulation and practical implementation. *Ocean dynamics* 53(4), 343–367.
- Fisher, M. and E. Andersson (2001). *Developments in 4D-Var and Kalman filtering*. European Centre for Medium-Range Weather Forecasts.
- for Aerology, W. M. O. C., A. Eliassen, and J. Bessemoulin (1960). *Upper Air Network Requirements for Numerical Weather Prediction: Report of a Working Group of the Commission for Aerology*. Secretariat of the World Meteorological Organization.
- Gandin, L. (1965). Objective analysis of meteorological fields gidrometeorologicheskoe izdatel'stvo, leningrad, 1960. *Translated from Russian into English, Israel Program for Scientific Translations, Jerusalem*, 242.
- Ginsberg, J., M. H. Mohebbi, R. S. Patel, L. Brammer, M. S. Smolinski, and L. Brilliant (2008). Detecting influenza epidemics using search engine query data. *Nature* 457(7232), 1012–1014.
- Gorman, M. and P. Widmann (1984). Chaotic flow regimes in a convection loop. *Phys. Rev. Lett.* (52), 2241–2244.
- Gorman, M., P. Widmann, and K. Robbins (1986). Nonlinear dynamics of a convection loop: a quantitative comparison of experiment with theory. *Physica D* (19), 255–267.
- Hamill, T. M., J. S. Whitaker, and C. Snyder (2001). Distance-dependent filtering of background error covariance estimates in an ensemble kalman filter. *Monthly Weather Review* 129(11), 2776–2790.
- Harris, K. D., E. H. Ridouane, D. L. Hitt, and C. M. Danforth (2011). Predicting flow reversals in chaotic natural convection using data assimilation. *arXiv preprint arXiv:1108.5685*.
- Hsiang, S. M., M. Burke, and E. Miguel (2013). Quantifying the influence of climate on human conflict. *Science* 341(6151).
- Hunt, B. R., E. J. Kostelich, and I. Szunyogh (2007). Efficient data assimilation for spatiotemporal chaos: A local ensemble transform kalman filter. *Physica D: Nonlinear Phenomena* 230(1), 112–126.
- Issa, R. I. (1986). Solution of the implicitly discretised fluid flow equations by operator-splitting. *Journal of Computational physics* 62(1), 40–65.
- Jiang, Y. and M. Shoji (2003). Spatial and temporal stabilities of flow in a natural circulation loop: influences of thermal boundary condition. *J Heat Trans.* (125), 612–623.
- Kalnay, E. (2003). *Atmospheric modeling, data assimilation, and predictability*. Cambridge university press.
- Kalnay, E., H. Li, T. Miyoshi, S.-C. YANG, and J. BALLABRERA-POY (2007). 4-d-var or ensemble kalman filter? *Tellus A* 59(5), 758–773.
- Keller, J. Periodic oscillations in a model of thermal convection.
- Kwant, W. and C. Boardman (1992). Prismliquid metal cooled reactor plant design and performance. *Nuclear engineering and design* 136(1), 111–120.
- Lorenc, A. (1981). A global three-dimensional multivariate statistical interpolation scheme. *Monthly Weather Review* 109(4), 701–721.
- Lorenc, A. C. (1986). Analysis methods for numerical weather prediction. *Quarterly Journal of the Royal Meteorological Society* 112(474), 1177–1194.

BIBLIOGRAPHY

- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the atmospheric sciences* 20(2), 130–141.
- Louisos, W. F., D. L. Hitt, and C. M. Danforth (2013). Chaotic flow in a 2d natural convection loop with heat flux boundaries. *International Journal of Heat and Mass Transfer* 61, 565–576.
- Navier, C. (1823). Mémoire sur les lois du mouvement des fluides. *Mémoires de l'Académie Royale des Sciences de l'Institut de France* 6, 389–440.
- Ott, E., B. R. Hunt, I. Szunyogh, A. V. Zimin, E. J. Kostelich, M. Corazza, E. Kalnay, D. Patil, and J. A. Yorke (2004). A local ensemble kalman filter for atmospheric data assimilation. *Tellus A* 56(5), 415–428.
- Patankar, S. V. and D. B. Spalding (1972). A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows. *International Journal of Heat and Mass Transfer* 15(10), 1787–1806.
- Purser, R. J. (1984). A new approach to the optimal assimilation of meteorological data by iterative bayesian analysis. In *Conference on Weather Forecasting and Analysis, 10 th, Clearwater Beach, FL*, pp. 102–105.
- Ridouane, E. H., C. M. Danforth, and D. L. Hitt (2010). A 2-d numerical study of chaotic flow in a natural convection loop. *International Journal of Heat and Mass Transfer* 53(1), 76–84.
- Saltzman, B. (1962). Finite amplitude free convection as an initial value problem-i. *Journal of the Atmospheric Sciences* 19(4), 329–341.
- Savely, R., B. Cockrell, , and S. Pines (1972). Apollo experience report - onboard navigational and alignment software. *Technical Report*.
- Sornette, D. and W.-X. Zhou (2006). Predictability of large future changes in major financial indices. *International Journal of Forecasting* 22(1), 153–168.
- Stokes, S. G. G. (1846). *On the Friction of Fluids in Motion, and the Equilibrium and Motion of Elastic Solids*.
- Talagrand, O. (1997). Assimilation of observations, an introduction. *JOURNAL-METEOROLOGICAL SOCIETY OF JAPAN SERIES 2* 75, 81–99.
- Thompson, P. (1990). Charney and the revival of nwp. *The Atmosphere - a challenge*, 93–119.
- Tippett, M. K., J. L. Anderson, C. H. Bishop, T. M. Hamill, and J. S. Whitaker (2003). Ensemble square root filters*. *Monthly Weather Review* 131(7), 1485–1490.
- Welander, P. (1995). On the oscillatory instability of a differentially heated fluid loop. *International Geophysics Series* 59.
- Yang, S.-C., D. Baker, H. Li, K. Cordes, M. Huff, G. Nagpal, E. Okereke, J. Villafane, E. Kalnay, and G. S. Duane (2006). Data assimilation as synchronization of truth and model: Experiments with the three-variable lorenz system*. *Journal of the atmospheric sciences* 63(9), 2340–2354.
- Yuen, P. and H. Bau (1999). Optimal and adaptive control of chaotic convection. *Phys. Fluids* (11), 1435–1448.

Appendix A: Parameters

Table A.1: EKF Covariance Inflation

Window Length	Additive Inflataion	Multiplicative Inflation
30s	0.05	0.15

Appendix B: Code

The core components of the code necessary to reproduce all results herein is as follows. None of OpenFOAM version 2.2.1 is included, as it is freely available for download online.

```
classdef OpenFOAM<handle
    %OpenFOAM: class for an OpenFOAM run
    %   Goal is to implement a class for which MATLAB can
    %   control and run an OpenFOAM instance
    %
    %   USAGE:
    %       -initialize an OpenFOAM directory, read to be run, at 'testCase'
    %       ens1 = OpenFOAM();
    %       ens1.init('testCase');
    %       -set the T variable for the initial run
    %       ens1.T = 300*ones(36000,1);
    %       -run OpenFOAM on this case
    %       ens1.run();
    %       -run more into the future
    %       ens1.ETIME = 40;
    %       ens1.run();
    %       -go back and read T from time 30 (note they need to be divisors of 5 now
    %       ens1.read(30)
```

APPENDIX B. CODE

```
properties
    foamLab = '/users/a/r/areagan/work/2013/data-assimilation/OpenFOAM/foamLab.sh';
    TIME=0;
    ETIME=20;
    DIR='testCase'
    FLUX=10000; % for flux BC
    BCbottom = '310'; % for fixed value BC
    BCtop = '290';
    BC = 'fixedValue';
    TURB = 'off';
    TURBMODEL = 'kEpsilon';
    T = 300*ones(36000,1);
    U = zeros(36000,3); %1e-6*ones(36000,3);
    p = zeros(36000,1); %1e-6*ones(36000,1);
    p_rgh = zeros(36000,1); %1e-6*ones(36000,1);
    Tstep = 1;
    WRITEp = 6;
    WRITEint = 20;
    VALUE = [];
    dataVec
end % properties

methods
    function OF = OpenFOAM(varargin)
        % intialize the class
    end
    function init(OF, varargin)
        if nargin > 1
            OF.DIR = varargin{1};
        end
    end
end
```

APPENDIX B. CODE

```

    end

    % initialize a a clean dir
    system(sprintf('%s -i -d %s',OF.foamLab,OF.DIR));
    % !OF.foamLab -i -d OF.DIR
    %OF.setFlux
end

function read(OF,time,varargin)

    % read in a specific time variable
    % first: have foamLab write it as a csv
    caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',OF.I);
    system(sprintf('%s -r %d -d %s',OF.foamLab,time,caseDir));
    OF.T = csvread(sprintf('%s/%d/T.csv',caseDir,time));
    OF.U = csvread(sprintf('%s/%d/U.csv',caseDir,time));
    OF.p = csvread(sprintf('%s/%d/p.csv',caseDir,time));
    OF.p_rgh = csvread(sprintf('%s/%d/p_rgh.csv',caseDir,time));
    %return T
end

function write(OF,time,varargin)

    % write out variable
    caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',OF.I);
    csvwrite(sprintf('%s/%d/T.csv',caseDir,time),OF.T);
    csvwrite(sprintf('%s/%d/U.csv',caseDir,time),OF.U);
    csvwrite(sprintf('%s/%d/p.csv',caseDir,time),OF.p);
    csvwrite(sprintf('%s/%d/p_rgh.csv',caseDir,time),OF.p_rgh);
    system(sprintf('%s -W %d -d %s',OF.foamLab,time,caseDir));
end

function slice(OF,num,varargin)

    % 'num' is the slice ID
    % this can be 1-8, representing the obviousness below

```

APPENDIX B. CODE

```

% the top slice Cell IDs from paraview
right = [35098 35156 35213 35268 35321 35372 35421 35468 35513 35556 35598];
oneThirty = 0:1:29;
top = 17218:58:18900;
tenThirty = 0:1:29;
left = [ 0 1 4 9 16 25 36 49 64 81 100 121 144 169 196 225 256 289 324 361];
sevenThirty = 0:1:29;
bottom = 17159:58:18841;
fourThirty = 0:1:29;
% add 1 bc matlab indexes at 1
allCellID = [right+1; oneThirty+1; top+1; tenThirty+1; left+1; sevenThirty+1];
%disp(allCellID)
cellID=allCellID(num,:);
%disp(cellID)
OF.VALUE = [mean(OF.T(cellID)) mean(OF.U(cellID,1)) mean(OF.U(cellID,2))];
end

function run(OF,varargin)

% convert relevant variables to foam-readable and insert them,
% if necessary
caseDir = sprintf('/users/a/r/areagan/OpenFOAM/areagan-2.2.1/run/%s',OF.TIME);
tOutFile = sprintf('%s/%d/%s',caseDir,OF.TIME,'T.csv');
csvwrite(tOutFile,OF.T);

% run the case
system(sprintf('%s -x -d %s -t %d -h %s -g %s -e %d -l %d -w %d -c %d -b %d',OF.CASE,OF.CASE,OF.TIME,OF.TIME,OF.TIME,OF.TIME,OF.TIME,OF.TIME,OF.TIME,OF.TIME,OF.TIME));
OF.TIME = OF.ETIME;

OF.T = csvread(sprintf('%s/%d/T.csv',caseDir,OF.ETIME));
end

```

APPENDIX B. CODE

```
function makeDataVec(OF, direction)

    switch direction

        case 'in' % into the datavec

            disp('converting the variables OF.T ... into OF.dataVec ');
            OF.dataVec = ones(36000*6,1); % column vector
            OF.dataVec(1:6:36000*6) = OF.T;
            OF.dataVec((1:6:36000*6)+1) = OF.U(:,1);
            OF.dataVec((1:6:36000*6)+2) = OF.U(:,2);
            OF.dataVec((1:6:36000*6)+3) = OF.U(:,3);
            OF.dataVec((1:6:36000*6)+4) = OF.p;
            OF.dataVec((1:6:36000*6)+5) = OF.p_rgh;

        case 'out' % out of dataVec into T,U,p,p_rgh

            if length(OF.dataVec) > 1 % check that something is there

                OF.T = OF.dataVec(1:6:36000*6);
                OF.U(:,1) = OF.dataVec((1:6:36000*6)+1);
                OF.U(:,2) = OF.dataVec((1:6:36000*6)+2);
                OF.U(:,3) = OF.dataVec((1:6:36000*6)+3);
                OF.p = OF.dataVec((1:6:36000*6)+4);
                OF.p_rgh = OF.dataVec((1:6:36000*6)+5);

            else

                disp('there is nothing in dataVec ... did you mean to put in ')

            end

        end

    end

end % methods

end % classdef
```